PROJET — SÉCURITÉ MOBILE

Rétro-ingénierie des Services Google Play

Analyse de la Migration d'Appareils Google

Technologies, Jumelage, Chiffrement et Échange de Clés

Étudiants : EL KADIRI BOUTCHICH RADIA

EL AOUNI GHIZLANE

DAANOUN AICHA

ABIZA FADWA

Encadrant : Pr. KOULALI MOHAMMED AMINE

Filière / Promotion : Sécurité Informatique et cyber sécurité – 4ème année

Année universitaire : 2025 – 2026

October 7, 2026

Contents

1 Introduction **4**

2 Mise en place : Rootage de l'Appareil **5**
2.1 Déverrouillage du bootloader 5
2.2 Rootage via Magisk 5
2.2.1 Téléchargement de Magisk et du firmware 5
2.2.2 Génération du firmware patché par Magisk 6
2.2.3 Flashage via Odin3 8

3 Identification et Extraction des Packages **10**
3.1 Recherche des packages concernés 10
3.2 Extraction des APKs 11
3.3 Découverte de `com.google.android.gms` 11

4 Étude du Manifeste et des Permissions **12**
4.1 Analyse de l'`AndroidManifest.xml` 12
4.2 Étude des permissions 12
4.2.1 Permissions significatives 12
4.3 Services déclarés dans le manifeste — technologies observées . . . 13

5 Analyse Statique Approfondie **15**
5.1 Étude de `UsbD2dMigrateService` — piste explorée 15
5.1.1 Pourquoi cette classe 15
5.2 Question 1 — Technologies utilisées (synthèse) 15

6 Question 2 — Processus de Jumelage **16**
6.1 Jumelage Android → Android 16
6.2 Jumelage iOS → Android : QR code, hotspot Wi-Fi et protocole Lockdown 18
6.2.1 Protocole Lockdown / usbmuxd 21

7 Question 3 — Chiffrement des Données **27**
7.1 Première approche : recherche du mot-clé `cipher` 27
7.2 Classe `elk` — chiffrement AES-GCM et AES-CBC 27
7.2.1 AES/GCM/NoPadding : le chiffrement principal 28
7.2.2 AES/CBC/NoPadding : compatibilité protocole Lockdown 29

7.3 Classe ggq — gestion des clés de session par AESWrap 30
7.3.1 Initialisation et parsing du flux de clés 30
7.3.2 Déchiffrement de la clé de session (`ggq.b()`) 32

8 Question 4 — Échange et Dérivation des Clés 34
8.1 Méthodologie 34
8.2 Génération des paires de clés 34
8.3 Accord de clés : ECDH 35
8.4 Dérivation de clé 36
8.5 Remarque sur Tink 37

9 Analyse Dynamique : Instrumentation avec Frida 38
9.1 Objectifs et méthodologie 38
9.2 Première tentative : hook direct de la classe `elk` 39
9.2.1 Hypothèse de départ 39
9.2.2 Résultat et contradiction 40
9.2.3 Diagnostic : collision de `ClassLoader` 40
9.3 Pivot stratégique : hook agnostique sur `javax.crypto.Cipher` . . . 42
9.3.1 Principe 42
9.4 Mécanisme d'autodéfense : crash lié aux permissions Bluetooth . . 43
9.5 Limite environnementale : « Lazy Loading » et sécurité GMS 45
9.5.1 Diagnostic 45
9.5.2 Cartographie de l'architecture par écoute passive (« Macro-Radar ») 45
9.6 Preuve de concept : validation du hook par injection contrôlée . . . 48
9.7 Synthèse de l'analyse dynamique 50
9.8 Limites de l'étude et perspectives 50

10 Conclusion Générale 52
10.1 Synthèse des réponses aux quatre questions 52
10.2 Perspectives : analyse dynamique avec Frida 52

A Analyse détaillée de `UsbD2dMigrateService` 54
A.0.1 Point de départ : la méthode `onCreate()` 54
A.0.2 Bloc 1 — Le mécanisme de log et la vérification du mode E2E 55
A.0.3 Bloc 2 — Configuration USB et conformité du câble (`e().c()`) 57
A.0.4 Bloc 3 — Annonce de disponibilité du service (`fiu.at(this)`) 58

A.0.5 Bloc 4 — Collecte des métadonnées de l'accessoire USB . . . 58
A.0.6 Bloc 5 — Feature flags et modes de transfert 59

1. Introduction

Ce rapport documente le processus et les résultats de notre analyse par rétro-ingénierie de l'outil de migration de données proposé par Google lors de la première configuration d'un appareil Android. Cet outil permet de transférer contacts, photos, applications et paramètres depuis un appareil source — Android ou iOS — vers un nouvel appareil Android.

L'objectif de ce projet est de lever le voile sur les mécanismes internes de cette migration, en nous concentrant spécifiquement sur quatre axes de sécurité et de confidentialité :

1. **Quelle technologie de transfert** est utilisée au niveau bas (Bluetooth, Wi-Fi, USB, QR code, ou une combinaison) ?
2. **Comment le jumelage s'effectue-t-il** entre les deux appareils, et comment se reconnaissent-ils ?
3. **Les données transmises sont-elles chiffrées** pendant le transfert ?
4. **Comment les clés de chiffrement sont-elles négociées et dérivées** ?

La méthodologie adoptée suit une progression sur plusieurs étapes :

- **Mise en place :** rootage du Samsung SM-A013G via Magisk et Odin, puis accès `root` par ADB.
- **Identification :** identification des packages pertinents, extraction et décompilation des APKs via JADX.
- **Analyse statique :** analyse statique approfondie du code décompilé — exploration du manifeste, étude des services, interprétation des classes obfusquées.

L'analyse dynamique avec Frida est conduite en parallèle et fera l'objet d'un complément à ce rapport.

2. Mise en place : Rootage de l'Appareil

Avant de pouvoir accéder aux APKs internes et utiliser ADB en mode root, il a fallu rooter notre appareil cible : un **Samsung Galaxy A01 Core (SM-A013G)**.

2.1 Déverrouillage du bootloader

Le bootloader est l'équivalent du BIOS pour les téléphones Android. Le déverrouiller est la première condition indispensable pour tout rootage. Sur un Samsung, la procédure est la suivante :

1. Activer le *mode développeur* (via les paramètres système) et activer l'option **OEM unlock**.
2. Entrer dans le *Download Mode* en maintenant simultanément les boutons volume +/− tout en branchant le câble USB au PC.
3. Un appui long sur volume + fait apparaître la page de confirmation *"Do you want to unlock the bootloader ?"*. Un appui court sur volume + confirme.
4. Le téléphone redémarre. Le bootloader est maintenant déverrouillé.

2.2 Rootage via Magisk

2.2.1 Téléchargement de Magisk et du firmware

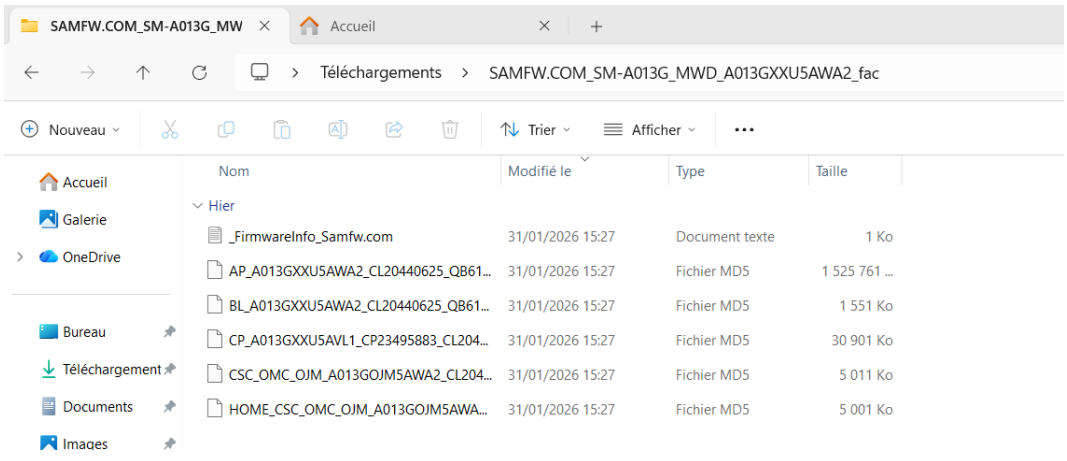
Magisk est l'outil standard pour rooter les appareils Android de manière non destructive. Il modifie l'image `boot` du firmware pour y injecter son daemon root, sans toucher à la partition système.

Magisk a été téléchargé depuis son dépôt officiel GitHub.[1] Le firmware officiel correspondant à notre appareil a été récupéré sur le site `samfw.com`.[2]

```
Modèle   : SM-A013G
Variante : MWD
Version  : A013GXXU5AWA2
```

[1] `https://github.com/topjohnwu/Magisk/releases`
[2] `samfw.com/firmware/SM-A013G/MWD/A013GXXU5AWA2`

Une fois l'archive ZIP extraite sur notre PC, les composants suivants sont présents :

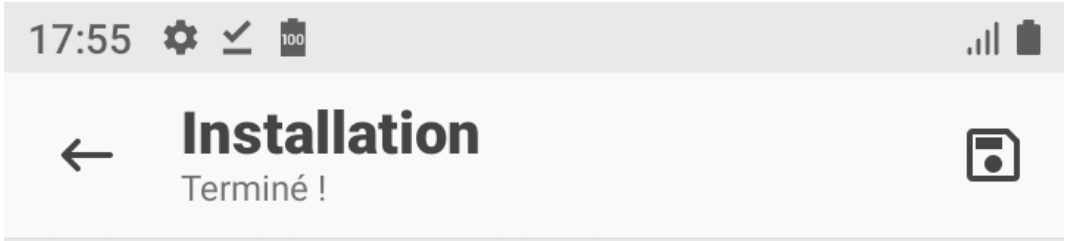


Remarque

Dans les versions anciennes de Magisk, il était nécessaire d'extraire les sous-fichiers de la partition **AP** (qui est elle-même une archive) pour accéder directement aux images `boot.img.lz4` et `recovery.img.lz4`. Dans les versions récentes, Magisk accepte directement le fichier AP complet.

2.2.2 Génération du firmware patché par Magisk

Depuis l'application Magisk installée sur le téléphone, on clique sur **Install**, puis on sélectionne le fichier **AP** du firmware d'origine. Magisk effectue alors les opérations suivantes : extraction de `boot.img`, vérification du ramdisk, patch du ramdisk, repack de l'image. En cas de succès, un message `All done!` apparaît et le fichier patché est enregistré dans `/storage/emulated/0/Download/`.




```
- Device platform: armeabi-v7a
- Installing: 30.6 (30600)
- Processing tar file
-- Extracting: boot.img
-- Copying   : recovery.img.lz4
-- Copying   : dtbo.img.lz4
-- Copying   : super.img.lz4
-- Skipping  : userdata.img.lz4
-- Copying   : spmfw-verified.img.lz4
-- Copying   : tee-verified.img.lz4
-- Patching  : vbmeta.img
-- Copying   : mcupmfw-verified.img.lz4
-- Copying   : meta-data/
-- Copying   : meta-data/fota.zip
- Unpacking boot image
- Checking ramdisk status
- Stock boot image detected
- Patching ramdisk
- Pre-init storage partition: cache
- Repacking boot image
-- Writing   : boot.img

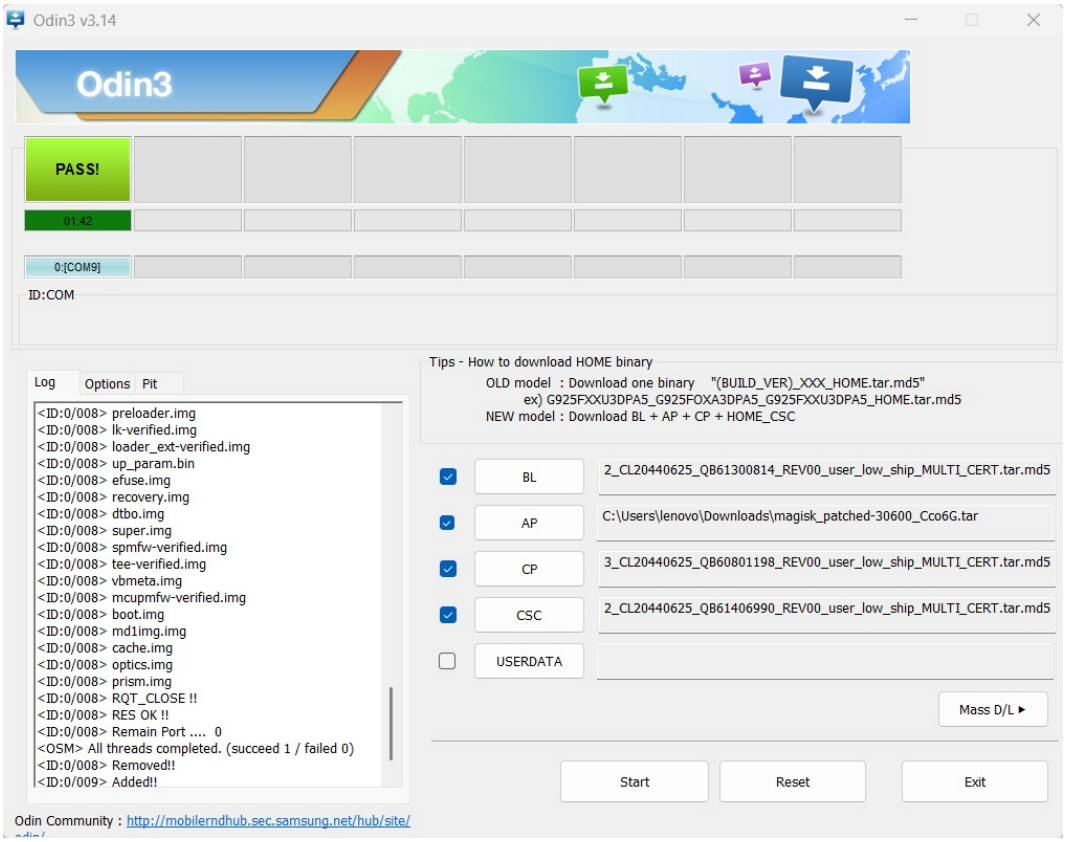
****************************
 Output file is written to
 /storage/emulated/0/Download/magisk_patched-30600_F
****************************
- All done!
```

2.2.3 Flashage via Odin3

Odin3 est l'outil officieux de maintenance avancée pour les appareils Samsung. Il permet de flasher manuellement un firmware complet ou partiel via USB, ce qui est précisément ce dont nous avons besoin pour injecter notre image `boot` patchée.[3]

Dans Odin, les fichiers sont assignés aux slots suivants :

- **BL, CP** : depuis le firmware d'origine.
- **CSC** : on sélectionne `HOME_CSC` (et non le `CSC` classique) — ce choix est crucial pour éviter la réinitialisation complète aux paramètres d'usine.
- **AP** : l'archive **patchée** par Magisk (et non l'AP d'origine).



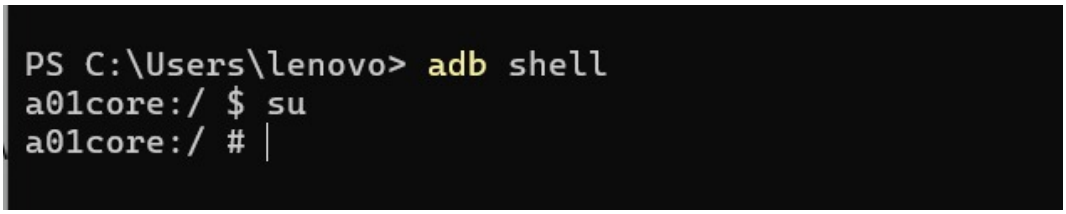
Attention

Il est impératif de **décocher l'option Auto Reboot** dans Odin avant de lancer. Si cette option reste activée, le processus échoue systématiquement.

[3] Tutoriels de référence : `https://www.youtube.com/watch?v=26YjDVjE0RM` et `https://www.youtube.com/watch?v=7ZzK9qBxv7M`

Pour entrer en mode Download depuis le mode bootloader déjà atteint, un simple appui sur le bouton volume + suffit. On clique ensuite sur **Start** dans Odin et on attend.

Normalement, à cette étape, le téléphone passe par un *Recovery Mode* avant de s'allumer, et Magisk affiche une sorte de setup post-installation. Dans notre cas, ces deux phases ne se sont pas déclenchées — mais le rootage a fonctionné quand même, ce qu'on vérifie en exécutant `su` via ADB :



Le prompt # confirme que l'accès root est bien obtenu. Le mode débogage ADB doit être activé dans les options développeur pour que cette connexion soit possible.

3. Identification et Extraction des Packages

3.1 Recherche des packages concernés

L'étude commence par l'identification des packages Google responsables de la migration. Depuis un shell ADB en root, on liste tous les packages Google présents sur l'appareil :

```
PS C:\Users\lenovo> adb shell
a01core:/ $ su
a01core:/ # pm list packages | grep google
package:com.google.android.ext.services
package:com.google.android.onetimeinitializer
package:com.google.android.ext.shared
package:com.google.android.apps.mapslite
package:com.google.android.apps.navlite
package:com.google.android.configupdater
package:com.google.android.overlay.modules.permissioncontroller
package:com.google.android.gm.lite
package:com.google.android.apps.tachyon
package:com.google.android.apps.searchlite
package:com.google.android.permissioncontroller
package:com.google.android.setupwizard
package:com.google.android.overlay.modules.ext.services
package:com.google.android.apps.wellbeing
package:com.google.android.apps.nbu.files
package:com.google.android.overlay.gmsconfig
package:com.google.android.modulemetadata
package:com.google.android.webview
package:com.google.android.syncadapters.contacts
package:com.google.android.packageinstaller
package:com.google.android.gms
package:com.google.android.gsf
package:com.google.android.tts
package:com.google.android.partnersetup
package:com.google.android.overlay.modules.permissioncontroller.forframework
package:com.google.android.feedback
package:com.google.android.printservice.recommendation
package:com.google.android.syncadapters.calendar
package:com.google.android.apps.speechservices
package:com.google.android.apps.photosgo
package:com.google.android.overlay.gmsagoconfig
package:com.google.android.apps.assistant
package:com.google.android.gms.location.history
package:com.google.android.apps.youtube.mango
package:com.google.android.apps.youtube.music
package:com.google.android.overlay.gmsgoconfig
package:com.google.android.inputmethod.latin
package:com.google.android.apps.restore
```

Le package qui nous intéresse prioritairement est `com.google.android.apps.restore`.
Un second package s'avérera crucial au fur et à mesure de l'analyse : `com.google.android.gms` (Google Mobile Services).

3.2 Extraction des APKs

Pour chacun des packages, on localise le chemin des APKs sur l'appareil via `pm path`, puis on les transfère sur notre PC via `adb pull` :

```
PS C:\Users\lenovo> adb pull /data/app/com.google.android.apps.restore-hM8T1b1VWZ5RIXbF-uobzw==/base.apk
/data/app/com.google.android.apps.restore-hM8T1b1VWZ5RIXbF...le pulled, 0 skipped. 12.8 MB/s (12983713 bytes in 0.
965s)
PS C:\Users\lenovo> adb pull /data/app/com.google.android.apps.restore-hM8T1b1VWZ5RIXbF-uobzw==/split_config.fr.ap
k
/data/app/com.google.android.apps.restore-hM8T1b1VWZ5RIXbF... file pulled, 0 skipped. 5.6 MB/s (192921 bytes in 0.
033s)
PS C:\Users\lenovo> adb pull /data/app/com.google.android.apps.restore-hM8T1b1VWZ5RIXbF-uobzw==/split_config.xhdpi
.apk
```

Les trois APKs récupérés sont :

Fichier	Taille	Date
`base.apk`	12 680 Ko	01/02/2026
`split_config.fr.apk`	189 Ko	01/02/2026
`split_config.xhdpi.apk`	87 Ko	01/02/2026

L'analyse commence par `base.apk`, le module principal. Les APKs sont ouverts avec **JADX**, un décompilateur Java/Android qui nous donne accès au code source approximatif ainsi qu'au `AndroidManifest.xml`.

Remarque

La plupart des imports et des noms de classes sont **obfusqués** (renommés en lettres courtes comme `cvh`, `dsh`, `cxq`, etc.). JADX garde cependant les noms originaux dans les logs et les commentaires lorsqu'ils sont disponibles, ce qui constitue notre principal outil de désobfuscation.

3.3 Découverte de `com.google.android.gms`

Au cours de l'analyse du package restore, on réalise rapidement que certains mécanismes importants — notamment la découverte des appareils et les opérations cryptographiques — sont délégués à GMS. On extrait donc également ce package :

```
1|a01core:/ $ pm path com.google.android.gms
package:/data/app/com.google.android.gms-Gv47PC7l0aVWcSoymx3Eqg==/base.apk
package:/data/app/com.google.android.gms-Gv47PC7l0aVWcSoymx3Eqg==/split_config.fr.apk
package:/data/app/com.google.android.gms-Gv47PC7l0aVWcSoymx3Eqg==/split_config.xhdpi.apk
a01core:/ $
```

4. Étude du Manifeste et des Permissions

4.1 Analyse de l'`AndroidManifest.xml`

Le manifeste de `com.google.android.apps.restore` (version `1.0.862715935`, SDK cible 36, minimum 28) constitue une première source structurée. Il expose les services et les permissions, ce qui donne rapidement une idée des capacités de l'application dans le contexte de la migration.

4.2 Étude des permissions

Avant d'entrer dans le détail, on remarque que certaines permissions sont conditionnées par `maxSdkVersion`. Par exemple, `ACCESS_FINE_LOCATION` est limitée à Android 12, car elle était auparavant nécessaire pour la découverte Wi-Fi/Bluetooth, remplacée ensuite par des permissions plus spécifiques comme `NEARBY_WIFI_DEVICES`. Ce point montre une adaptation aux évolutions du modèle de sécurité Android.

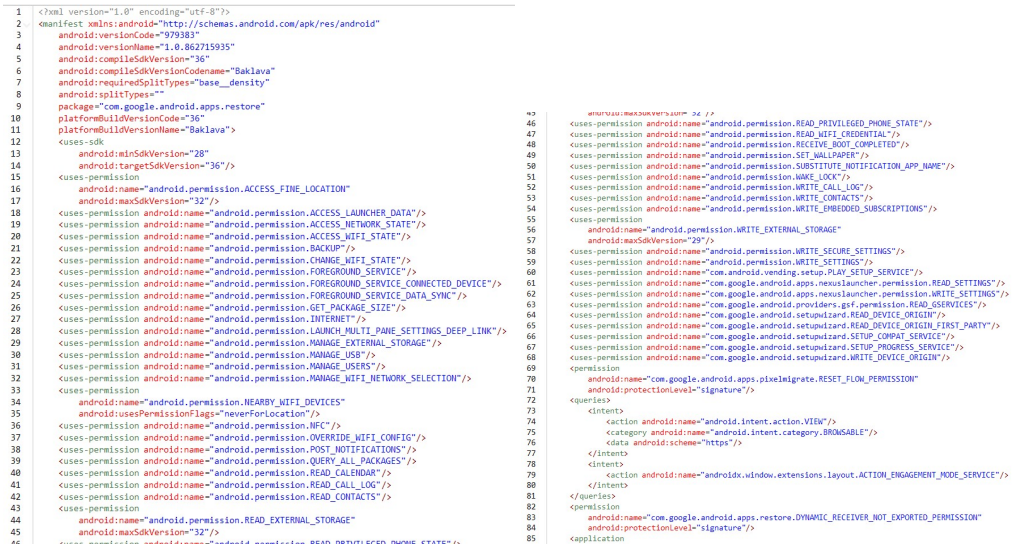


Figure 4.1: Liste des permissions

4.2.1 Permissions significatives

On ne détaille ici que les permissions pertinentes pour notre analyse :

`NEARBY_WIFI_DEVICES` — utilisée pour la découverte d'appareils à proximité sans

accès à la localisation (Android 13+), cohérente avec Nearby Connections.

`MANAGE_EXTERNAL_STORAGE` — permet d'accéder aux fichiers d'autres applications, nécessaire pour récupérer les données à migrer.

`MANAGE_USB` — indique un support de transfert via USB sans interaction utilisateur répétée.

`ACCESS_NETWORK_STATE` et `ACCESS_WIFI_STATE` — permettent d'observer et gérer l'état réseau pendant la migration.

`READ_CONTACTS`, `READ_CALL_LOG`, `READ_CALENDAR` — montrent que l'application accède directement aux données utilisateur à transférer.

`QUERY_ALL_PACKAGES` — permet d'énumérer les applications installées, utile pour reconstruire l'environnement sur le nouvel appareil.

`MANAGE_USERS` — suggère la gestion de profils multiples (ex : profil professionnel).

`ACCESS_LAUNCHER_DATA` — permission de type signature, permettant de récupérer la configuration du launcher (écran d'accueil, widgets, etc.).

Globalement, les permissions observées sont cohérentes avec un outil système de migration : accès large aux données, contrôle des interfaces de communication (Wi-Fi, USB), et intégration profonde avec le système Android.

On note également la présence de permissions de niveau signature ou système, ce qui confirme que l'application est préinstallée et dispose de privilèges élevés.

4.3 Services déclarés dans le manifeste — technologies observées

La partie la plus exploitable du manifeste pour répondre à la question des technologies est la liste des services. On y trouve quatre services de migration déclarés comme `foregroundService` de type `connectedDevice` :

```
<service
    android:name="...migrate.component.UsbD2dMigrateService"
    android:exported="true"
    android:foregroundServiceType="connectedDevice"/>
<service
    android:name="...migrate.component.WifiD2dMigrateService
   "
    android:foregroundServiceType="connectedDevice"/>
```

```
8  <service
9      android:name="...migrate.component.
       MultiTransportD2dMigrateService"
10     android:foregroundServiceType="connectedDevice"/>
11 <service
12     android:name="...migrate.component.
       MultiTransportD2dMigrateServiceV2"
13     android:foregroundServiceType="connectedDevice"/>
```

La nomenclature *D2D* (Device-to-Device) indique que le transfert est direct entre les deux appareils. Les services correspondent à différents modes :

- `UsbD2dMigrateService` : transfert via USB
- `WifiD2dMigrateService` : transfert via Wi-Fi
- `MultiTransportD2dMigrateService` (+ V2) : orchestration multi-transport

On note aussi que `UsbD2dMigrateService` est déclaré `exported="true"`, ce qui signifie qu'il peut être déclenché depuis l'extérieur.

À ce stade, on a déjà une première réponse claire : la migration repose sur plusieurs canaux (USB et Wi-Fi), avec une couche d'orchestration au-dessus.

5. Analyse Statique Approfondie

5.1 Étude de `UsbD2dMigrateService` — piste explorée

5.1.1 Pourquoi cette classe

On a analysé la classe USB dans l'espoir d'y trouver des éléments sur le chiffrement ou le jumelage. C'était une piste logique vu que USB est un canal direct.

Cependant, on n'a rien trouvé d'exploitable à ce niveau. La structure du cycle de vie (onCreate, onStart, etc.) n'était pas claire, et le code est fortement obfusqué et dispersé.

On a donc choisi de ne pas pousser plus loin cette piste (ni les classes WiFi / multi-transport dans ce sens), puisqu'elle ne répondait pas directement à ce qu'on cherchait.

Le détail de cette exploration est conservé en annexe A.

5.2 Question 1 — Technologies utilisées (synthèse)

La réponse découle directement de l'observation du manifeste :

- USB : via `UsbD2dMigrateService`
- Wi-Fi : via `WifiD2dMigrateService`
- Multi-transport : via les services `MultiTransport*`

Ces services étant déclarés comme `foregroundService` de type `connectedDevice`, cela confirme leur rôle dans une migration directe entre appareils.

La suite de l'analyse s'est donc orientée vers les mécanismes de jumelage et de chiffrement, présentés dans les sections suivantes.

6. Question 2 — Processus de Jumelage

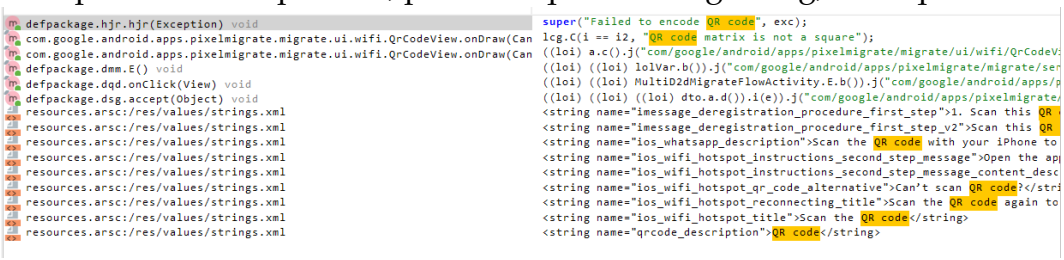
6.1 Jumelage Android → Android

Pour le Jumelage Android → Android : "Google Nearby Connections API utilisée"

Résultat de la recherche du mot clé "NearbyConnections" :

La première chose qu'on a pensé à chercher est l'API standard responsable du jumelage et du transfert des données, qui est Nearby Connections. On a trouvé des résultats sous forme de logs dont l'origine est le package GMS.

Une fois cela identifié, on a cherché un mot-clé plus général lié à la configuration, à savoir : "DiscoveryOptions". On a trouvé une classe (gom) qui contient plusieurs composants, parmi lesquels les flags f et g, ainsi qu'un tableau b.



La fonction toString(), qui est généralement utilisée pour le débogage et qui expose explicitement les variables sous forme de chaînes de caractères, nous a permis de comprendre le rôle de ces variables.

```
1    public final String toString() {
2
3        return String.format(Locale.US, "DiscoveryOptions{
     strategy: %s, discoveryMediums: %s, powerLevel: %d,
     analyticOptions: %s, uwbOptions: %s,
     allowBluetoothRadioToggling: %b, allowWifiRadioToggling:
     %b, fastAdvertisementServiceUuid: %s, dataElementsFilters
     : %s}", this.a, Arrays.toString(this.b), Integer.valueOf(
     this.c), this.d, this.e, Boolean.valueOf(this.f), Boolean
     .valueOf(this.g), this.h, this.i);
4    }
```

Listing 6.1: **La fonction toString() de gom **

On observe que : - le tableau b correspond à "discoveryMediums", donc aux moyens de découverte utilisés, - les flags f et g correspondent à : - allowBluetoothRadioToggling - allowWifiRadioToggling

Ces noms indiquent simplement que l'application peut activer automatiquement Bluetooth et WiFi si nécessaire.

On ne peut pas affirmer directement comment le tableau b est utilisé ni si les technologies sont combinées à ce niveau précis, mais la présence conjointe de ces éléments dans la configuration est cohérente avec le fonctionnement documenté de Nearby.

Ainsi, on peut conclure que : - Bluetooth est utilisé pour la phase de découverte (probablement BLE), - WiFi est utilisé pour la connexion et le transfert (probablement WiFi Direct).

Cette combinaison correspond au modèle classique de Nearby Connections : utiliser Bluetooth pour détecter les appareils à proximité, puis basculer vers une connexion WiFi pour le transfert de données, afin de bénéficier à la fois de la portée et du débit.

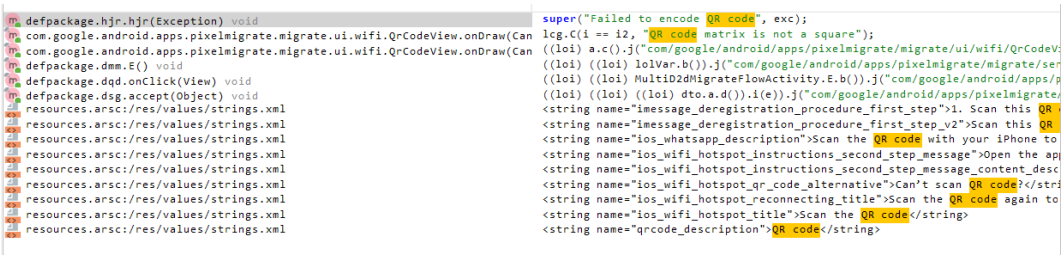
1

[1] Documentation Nearby : https://developers.google.com/nearby/connections/overview?hl=frestablishing_connect

6.2 Jumelage iOS → Android : QR code, hotspot Wi-Fi et protocole Lockdown

La migration depuis un iPhone nécessite un mécanisme spécifique car iOS ne supporte pas Wi-Fi Direct. L'analyse du code nous a menés à la classe `dsh` (dont le nom original est `IosWifiD2dHotspotFragment`), qui orchestre la création d'un hotspot Wi-Fi sur l'appareil Android et l'affichage d'un QR code contenant les paramètres de connexion.

La recherche du terme `"qr code"` dans JADX mène directement à la classe `QrCodeView` et à une exception `hjr` qui lui est associée (*"Failed to encode QR code"*). En cherchant les usages de la méthode `b()` de `QrCodeView` (son setter), on remonte vers la classe `dsh`.



JADX révèle que le nom original de la classe `dsh` est `IosWifiD2dHotspotFragment`. Ce nom est un condensé très parlant : c'est un Fragment (composant UI Android) dédié à la migration iOS (`Ios`), via Wi-Fi (`Wifi`), en mode D2D (`D2d`), créant un Hotspot (`Hotspot`).

```
private final void f(Optional optional, final boolean z) {
    optional.ifPresent(new Consumer() { // from class: dsg
        @Override // java.util.function.Consumer
        public final void accept(Object obj) {
            String strB;
            nbl nblVar;
            int i;
            int i2;
            int i3;
            int i4;
            dss dssVar = (dss) obj;
            if (z) {
                dssVar.getClass();
                String strI = niv.a.co() i();
                strB = strI.toLowerCase(Locale.US).contains("dogfoody") ? String.format("%s?%s", strI, dto.c(dssVar)) : dto.b(dto.c(dssVar));
            } else {
                dssVar.getClass();
                strB = dto.b(dto.c(dssVar));
            }
            dsh dshVar = this.a;
            Context contextZ = dshVar.z();
            int iS = jpc.a;
            if (jly.s(contextZ) && njo.c()) {
                QrCodeView.a(dshVar.e, dshVar.B());
            }
            if (jly.s(dshVar.z()) || nim.L()) {
                dshVar.d.setVisibility(8);
                dshVar.e.setVisibility(0);
                dshVar.b.b(strB);
                return;
            }
```

Dans la méthode `f(Optional optional, final boolean z)`, on trouve la logique de construction du QR code :

```
1 // z = true si mode "dogfoody" (mode test Google), false en
```

```
    mode normal
strB = strI.toLowerCase(Locale.US).contains("dogfoody")
    ? String.format("%s?%s", strI, dto.c(dssVar))  // mode
    test
    : dto.b(dto.c(dssVar));                          // mode
    normal
```

Listing 6.2: Construction du contenu du QR code

La variable z permet de distinguer le **mode dogfoody** (un mode de test interne Google) du mode normal. Le mot `"dogfoody"` — une référence à la pratique *dogfooding* (tester ses propres produits en interne) — est hardcodé dans le code, un indice rare sur les pratiques internes de Google.

En mode normal, `dto.b(dto.c(dssVar))` produit le contenu du QR code. En remontant dans la classe `dto` et la classe `dss` :

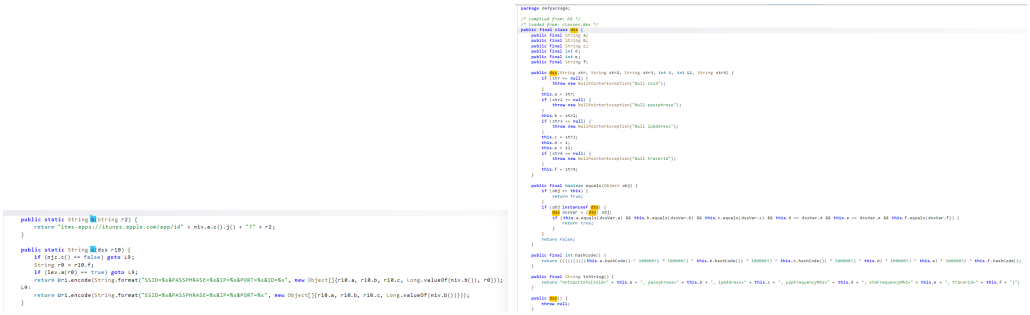


Figure 6.1: capture des classes dto et dds

`dto.c(dss)` sérialise les paramètres du hotspot en une chaîne URL encodée :

`SSID=...&PASSPHRASE=...&IP=...&PORT=...&ID=...`

`dto.b(uri)` formate cette chaîne en un lien App Store iOS :

`itms-apps://itunes.apple.com/app/id[ID]?[params]`

Ce lien est ensuite fourni à `QrCodeView.b(strB)`, qui génère et affiche le QR code correspondant.

```
@Override // android.view.View
public final void onDraw(Canvas canvas) throws hjr {
    canvas.getClass();
    super.onDraw(canvas);
    if (this.b != null) {
        try {
            hjq hjqVar = this.c;
            if (hjqVar == null) {
                opf.b("qrCodeRenderer");
                hjqVar = null;
            }
            String str = this.b;
            int width = getWidth();
            khj khjVar = hjqVar.g;
            str.getClass();
            try {
                nbl nblVar = nbm.b(str, (nbg) khjVar.a, khjVar.b).e;
                int i = nblVar.b;
                int i2 = nblVar.c;
                lcg.C(i == i2, "QR code matrix is not a square");
                int i3 = hjqVar.f;
                int iFloor = (int) Math.floor((width - r9) / i);
                int i4 = (iFloor * i) + i3 + i3;
                Bitmap bitmapCreateBitmap = Bitmap.createBitmap(i4, i4, Bitmap.Config.ARGB_8888);
                Canvas canvas2 = new Canvas(bitmapCreateBitmap);
                hjs hjsVar = hjqVar.e;
                if (hjsVar != null) {
                    hjsVar.a(canvas2, iFloor);
                } else {
                    canvas2.drawColor(hjqVar.a);
                }
                canvas2.save();
                float f = i3;
                canvas2.translate(f, f);
                for (int i5 = 0; i5 < i; i5++) {
                    for (int i6 = 0; i6 < i2; i6++) {
                        if (nblVar.a(i5, i6) == 1 && ((i5 > 7 || i6 > 7) && ((i5 < i - 8 || i6 > 7) && (i5 > 7 || i6 < i2 - 8)))) {
                            hjqVar.b.a(canvas2, i5, i6, iFloor);
                        }
                    }
                }
                hjv hjvVar = hjqVar.c;
                hjvVar.a(canvas2, 0, 0, iFloor);
                hjvVar.a(canvas2, 0, i2 - 7, iFloor);
                hjvVar.a(canvas2, i - 7, 0, iFloor);
                hjx hjxVar = hjqVar.d;
                canvas2.restore();
                if (!nim.a.co().z()) {
                    canvas.drawBitmap(bitmapCreateBitmap, 0.0f, 0.0f, (Paint) null);
                } else {
                    float width2 = (getWidth() - bitmapCreateBitmap.getWidth()) / 2.0f;
                    canvas.drawBitmap(bitmapCreateBitmap, width2, width2, (Paint) null);
                }
            } catch (naz e) {
                throw new hjr(e);
            }
        } catch (hjr e2) {
            ((loi) a.c().j("com/google/android/apps/pixelmigrate/migrate/ui/wifi/QrCodeView", "onDraw", 72, "QrCodeView.kt")).v("Failed to render QR code: ", e2);
        }
```

La classe `dss` est le conteneur des informations du hotspot, avec validation de chaque champ (exception si null) :

```
public dss(String ssid, String passphrase, String ipAddress,
           int port, int staFrequencyMHz, String tracerId) {
    if (str == null) throw new NullPointerException("Null ssid");
    this.a = str;     // ssid
    if (str2 == null) throw new NullPointerException("Null passphrase");
    this.b = str2;    // passphrase
    // ... ip, port, traceId
}
```

Listing 6.3: Classe dss — conteneur HotspotInfo

Conclusion

Pour iOS → Android : l'appareil Android crée un hotspot Wi-Fi dédié. Les paramètres de connexion (SSID, passphrase, IP, port) sont encodés dans un QR code, lequel contient également un deeplink vers l'application *Switch to Android* sur l'App Store iOS. L'iPhone scanne ce QR code, télécharge l'application,

et se connecte au hotspot Android comme un client Wi-Fi ordinaire. Le problème fondamental que résout ce mécanisme est simple : iOS ne supporte pas Wi-Fi Direct, donc il faut passer par un hotspot classique, et le QR code résout la question de comment communiquer les identifiants de connexion.

6.2.1 Protocole Lockdown / usbmuxd

Une fois le réseau Wi-Fi établi, le transfert effectif des données (qui inclut le jumelage) s'appuie sur le protocole **Lockdown / usbmuxd** d'Apple, que nous avons analysé en profondeur.

La recherche du terme `"pairing"` dans JADX mène principalement vers la classe `cxq`, dont le chemin de log révèle le nom original : `com.google.android.apps.pixelmigrate.ios.p`



C'est une **machine à états** dédiée au processus de jumelage iOS. Voici les classes qui gravitent autour d'elle :

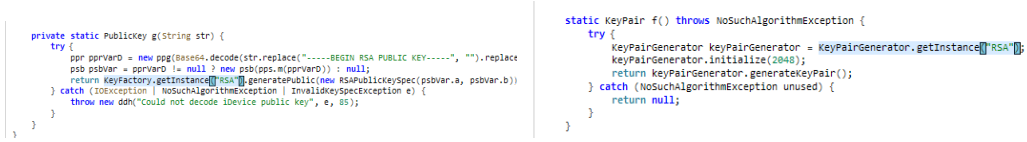


Figure 6.2: f() et g() de ddj

Génération et structure des certificats

La classe `ddj` est le générateur de certificats RSA. Deux fonctions clés ont été identifiées :

- `f()` : génère une paire de clés RSA-2048 via `KeyPairGenerator.getInstance("RSA")` et initialise le générateur avec 2048 bits.
- `g()` : décode la clé publique de l'iPhone (au format PEM Base64, délimité par `"---BEGIN RSA PUBLIC KEY---"`) en un objet `PublicKey` Java utilisable.

La classe `ddi` est le conteneur des trois certificats générés :

[2]Le protocole Lockdown est le protocole Apple permettant la communication entre iOS et un hôte (PC, Mac) via USB. Android joue ici le rôle d'un hôte. **usbmuxd** est le daemon Apple gérant le multiplexage TCP sur USB.

- `a` : certificat root (CA racine de la session)
- `b` : certificat device (de l'iPhone)
- `c` : certificat host (de l'appareil Android)

Ces certificats sont stockés au format DER, puis convertis en PEM pour leur envoi.

Construction de la requête de jumelage (`cxq.j()`)

La méthode `j()` de `cxq` construit la requête de jumelage. Son contenu est un dictionnaire **Plist** (Property List), le format de données structurées standard d'Apple (l'équivalent de JSON pour l'écosystème iOS). Ce choix est cohérent avec le fait qu'Android se présente comme un hôte PC aux yeux de l'iPhone.

```
    private final String j() {
        cks cksVar = new cks();
        cksVar.m("Label", "usbmuxd");
        cks cksVar2 = new cks();
        try {
            ddi ddiVar = this.g.b;
            cksVar2.put("DeviceCertificate", new ckp(Base64.encodeToString(ddj.a("CERTIFICATE", ddiVar.b.getEncoded()).getBytes(StandardCharsets.UTF_8), 0)));
            cksVar2.put("HostCertificate", new ckp(Base64.encodeToString(ddj.a("CERTIFICATE", ddiVar.c.getEncoded()).getBytes(StandardCharsets.UTF_8), 0)));
            cksVar2.put("RootCertificate", new ckp(Base64.encodeToString(ddj.a("CERTIFICATE", ddiVar.a.getEncoded()).getBytes(StandardCharsets.UTF_8), 0)));
            cksVar2.m("SystemBUID", C("system_buid"));
            cksVar2.m("HostID", C("host_id"));
            cksVar.put("PairRecord", cksVar2);
            cksVar.m("Request", "Pair");
            cksVar.m("ProtocolVersion", "2");
            cks cksVar3 = new cks();
            cksVar3.m("ExtendedPairingErrors", true);
            cksVar.put("PairingOptions", cksVar3);
            return v(cksVar.r());
        } catch (IOException e) {
            throw new cxr("IOException in getPairingRequest", e, 33);
        } catch (CertificateEncodingException e2) {
            throw new cxr("Error encoding the certificate as PEM in getPairingRequest", e2, 32);
        }
    }
```

Le dictionnaire Plist contient :

- **Label** : `"usbmuxd"` — identifiant du service selon la convention Lockdown
- **Request** : `"Pair"` — type de la requête
- **ProtocolVersion** : `"2"`
- **PairRecord** : sous-dictionnaire avec les trois certificats (`DeviceCertificate`, `HostCertificate`, `RootCertificate`) encodés en Base64, plus `SystemBUID` et `HostID` lus depuis les `SharedPreferences` Android.
- **PairingOptions** : sous-dictionnaire avec `ExtendedPairingErrors = true` pour obtenir des messages d'erreur détaillés.

Remarque

Les méthodes `put()` et `m()` de la classe `cks` servent toutes deux à ajouter des entrées clé-valeur au dictionnaire Plist. `m` est probablement une version

> obfusquée de `put` avec une nuance mineure (peut-être liée à la sérialisation), mais leur rôle fonctionnel est identique dans ce contexte.

Le résultat est converti via la méthode `v()` (héritée de la classe mère `cxw`) qui ajoute un caractère nul `\u0000` à la fin de la chaîne — un héritage des protocoles réseau en C comme `usbmuxd`.

Traitement des réponses : la méthode `c()`

La méthode `c(cxi, ByteBuffer r22, ByteBuffer r23)` est la machine à états centrale du protocole de jumelage.

Un `ByteBuffer` en Java est un outil de gestion de la mémoire tampon : il correspond à une région de la RAM allouée pour gérer le flux de données entrant et sortant, permettant de synchroniser la vitesse de réception avec le traitement. Chaque `ByteBuffer` contient un tableau d'octets sur lequel on peut effectuer des opérations de lecture/écriture de façon efficace.

`c()` fait le tour suivant :

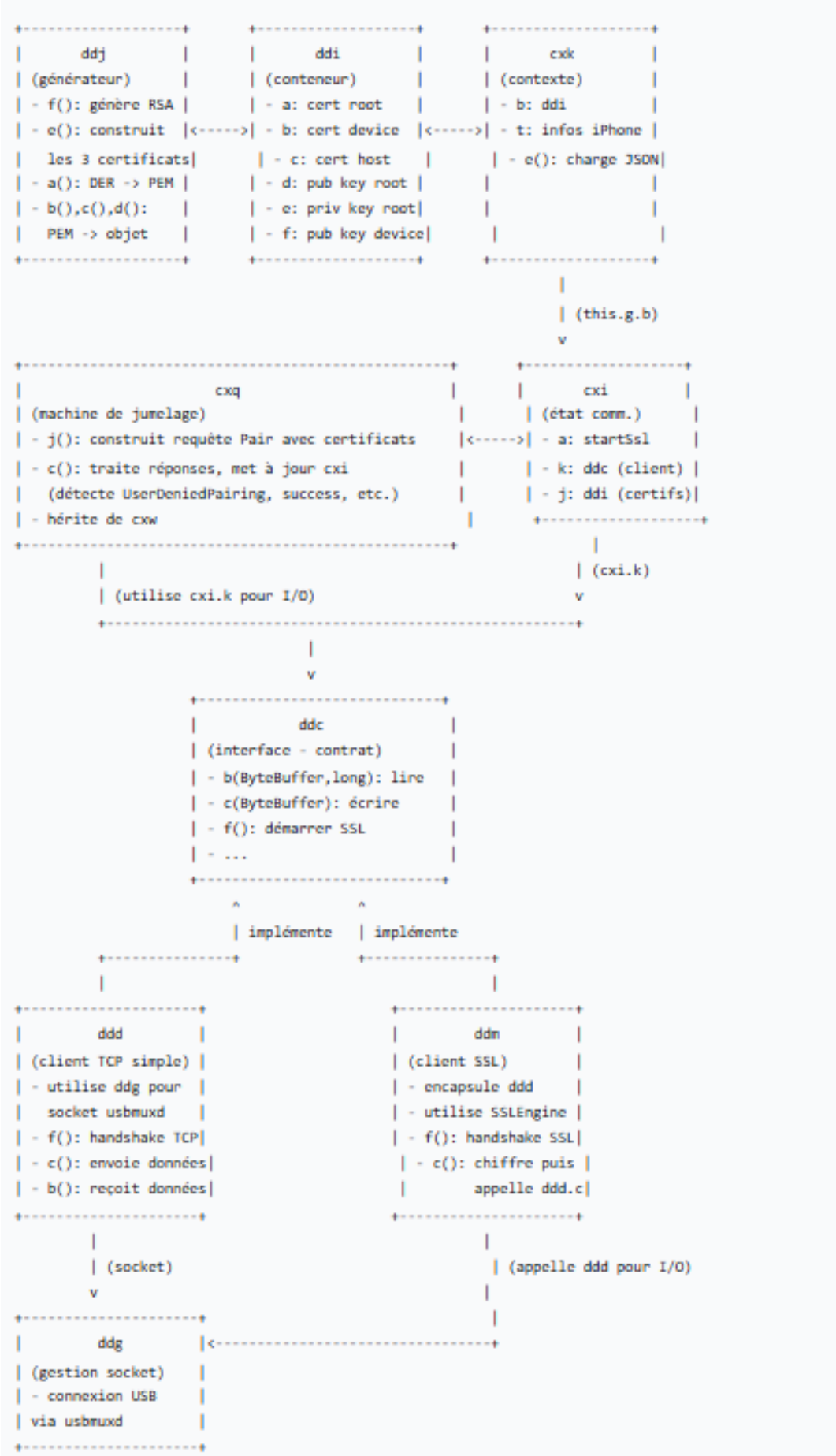
1. Lit les données reçues de l'iPhone (depuis `ByteBuffer r22`).
2. Les convertit en dictionnaire Plist via `k()`.
3. Analyse la réponse :
    - Si `UserDeniedPairing` → mise à jour de `cxiVar` pour signaler l'échec, arrêt.
    - Si réponse positive → préparation de l'étape suivante.
4. Modifie `cxiVar` en conséquence (ex : met `startSsl = true` si tout va bien).
5. Construit la réponse appropriée dans un `ByteBuffer` et la retourne.

En résumé : `c()` reçoit un message, l'interprète, change l'état, et renvoie la réponse appropriée. C'est la définition exacte d'une machine à états.

Phase de validation : défi-réponse RSA

Après le `Pair` initial, une phase de `ValidatePair` intervient. Elle est gérée dans la classe mère `cxw` et repose sur un échange de **défi-réponse** (challenge-response) basé sur RSA. Ce mécanisme garantit que chaque partie détient bien la clé privée

correspondant au certificat qu'elle a présenté — c'est une preuve de possession.


ddj
(générateur)
- f(): génère RSA
- e(): construit les 3 certificats
- a(): DER -> PEM
- b(),c(),d(): PEM -> objet
ddi
(conteneur)
- a: cert root
- b: cert device
- c: cert host
- d: pub key root
- e: priv key root
- f: pub key device
cxk
(contexte)
- b: ddi
- t: infos iPhone
- e(): charge JSON
(this.g.b)
cxq
(machine de jumelage)
- j(): construit requête Pair avec certificats
- c(): traite réponses, met à jour cxi (détecte UserDeniedPairing, success, etc.)
- hérite de cxw
cxi
(état comm.)
- a: startSsl
- k: ddc (client)
- j: ddi (certifs)
(cxi.k)
(utilise cxi.k pour I/O)
ddc
(interface - contrat)
- b(ByteBuffer,long): lire
- c(ByteBuffer): écrire
- f(): démarrer SSL
- ...
implémente
implémente
ddd
(client TCP simple)
- utilise ddg pour socket usbmuxd
- f(): handshake TCP
- c(): envoie données
- b(): reçoit données
ddm
(client SSL)
- encapsule ddd
- utilise SSLEngine
- f(): handshake SSL
- c(): chiffre puis appelle ddd.c
(socket)
(appelle ddd pour I/O)
ddg
(gestion socket)
- connexion USB via usbmuxd

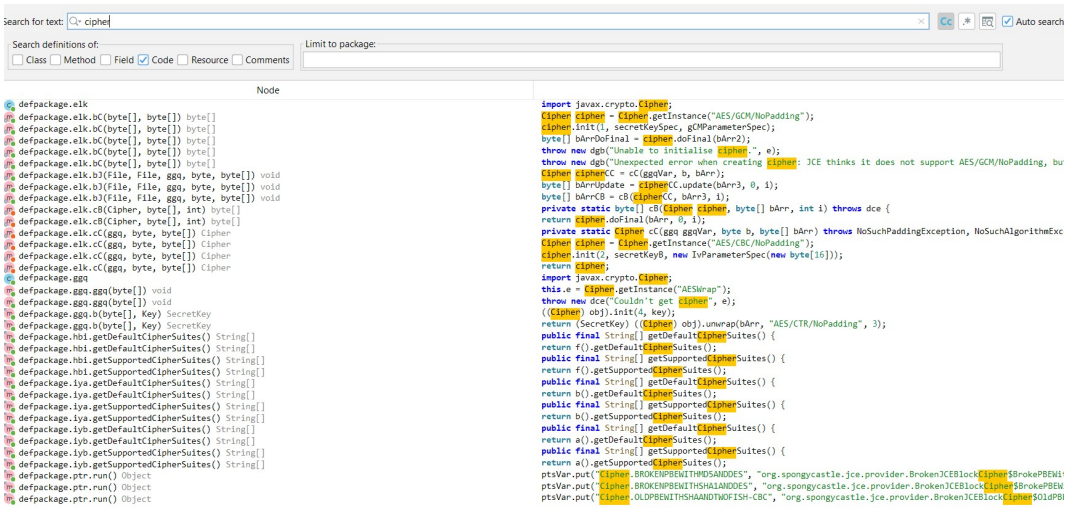
Conclusion

Pour iOS → Android : le jumelage repose sur le protocole **Lockdown / usbmuxd** d'Apple, qui simule un réseau TCP sur USB. Android joue le rôle d'un ordinateur hôte. Le processus se déroule en deux phases : (1) échange d'une requête `Pair` contenant trois certificats RSA en format Plist, suivie d'une vérification de réponse de l'iPhone ; (2) si accepté, une phase `ValidatePair` avec un défi-réponse RSA pour prouver la possession des clés privées. Si les deux phases sont réussies, le flag `startSsl = true` est positionné et la phase de chiffrement commence.

7. Question 3 — Chiffrement des Données

7.1 Première approche : recherche du mot-clé `cipher`

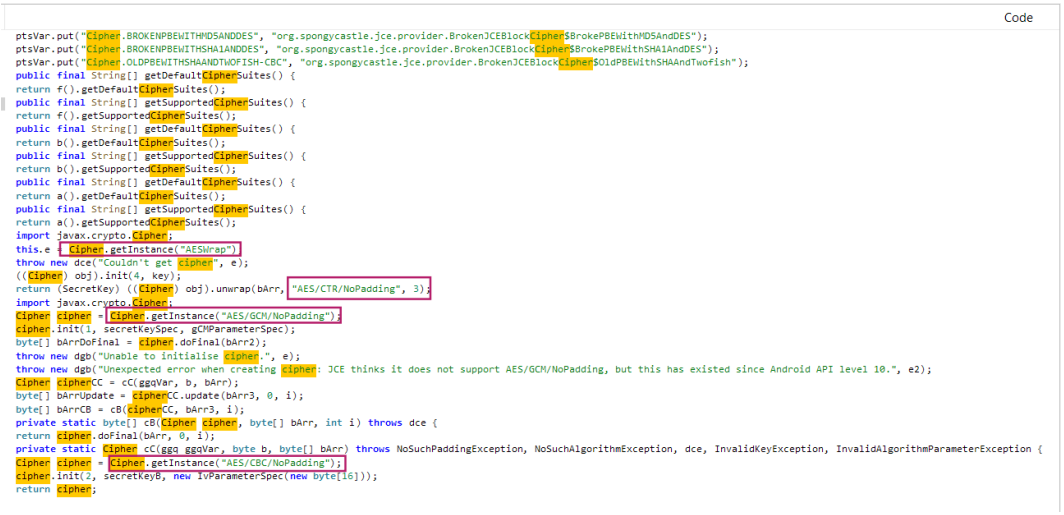
La recherche du terme `cipher` dans JADX retourne plus de 30 occurrences. On les observe :



La plupart concernent des algorithmes de la bibliothèque **Bouncy Castle** (présence de `spongycastle` dans les noms de packages). Parmi tous les résultats, quatre se démarquent et sont directement pertinents pour répondre à la question du chiffrement des données en transit. Ils sont répartis entre deux classes : `elk` et ggq.

7.2 Classe `elk` — chiffrement AES-GCM et AES-CBC

7.2.1 AES/GCM/NoPadding : le chiffrement principal



```
Code
ptsVar.put("Cipher.BROKENPBEWITHMD5ANDDES", "org.spongycastle.jce.provider.BrokenJCEBlockCipher$BrokePBEWithMD5AndDES");
ptsVar.put("Cipher.BROKENPBEWITHSHA1ANDDES", "org.spongycastle.jce.provider.BrokenJCEBlockCipher$BrokePBEWithSHA1AndDES");
ptsVar.put("Cipher.OLDPBEWITHSHAANDTWOFISH-CBC", "org.spongycastle.jce.provider.BrokenJCEBlockCipher$OldPBEWithSHAAndTwofish");
public final String[] getDefaultCipherSuites() {
return f().getDefaultCipherSuites();
public final String[] getSupportedCipherSuites() {
return f().getSupportedCipherSuites();
public final String[] getDefaultCipherSuites() {
return b().getDefaultCipherSuites();
public final String[] getSupportedCipherSuites() {
return b().getSupportedCipherSuites();
public final String[] getDefaultCipherSuites() {
return a().getDefaultCipherSuites();
public final String[] getSupportedCipherSuites() {
return a().getSupportedCipherSuites();
import javax.crypto.Cipher;
this.e = Cipher.getInstance("AESWrap");
throw new dce("Couldn't get cipher", e);
((Cipher) obj).init(4, key);
return (SecretKey) ((Cipher) obj).unwrap(bArr, "AES/CTR/NoPadding", 3);
import javax.crypto.Cipher;
Cipher cipher = Cipher.getInstance("AES/GCM/NoPadding");
cipher.init(1, secretKeySpec, gCMParameterSpec);
byte[] bArrDoFinal = cipher.doFinal(bArr2);
throw new dgb("Unable to initialise cipher.", e);
throw new dgb("Unexpected error when creating cipher: JCE thinks it does not support AES/GCM/NoPadding, but this has existed since Android API level 10.", e2);
Cipher cipherCC = cC(ggqVar, b, bArr);
byte[] bArrUpdate = cipherCC.update(bArr3, 0, i);
byte[] bArrCB = cB(cipherCC, bArr3, i);
private static byte[] cB(Cipher cipher, byte[] bArr, int i) throws dce {
return cipher.doFinal(bArr, 0, i);
private static Cipher cC(ggq ggqVar, byte b, byte[] bArr) throws NoSuchPaddingException, NoSuchAlgorithmException, dce, InvalidKeyException, InvalidAlgorithmParameterException {
Cipher cipher = Cipher.getInstance("AES/CBC/NoPadding");
cipher.init(2, secretKeyB, new IvParameterSpec(new byte[16]));
return cipher;
```

```java
public static byte[] bC(byte[] bArr, byte[] bArr2)
    throws BadPaddingException, NoSuchPaddingException,
    IllegalBlockSizeException {

    SecretKeySpec secretKeySpec = new SecretKeySpec(bArr, "
    AES");

    byte[] bArr3 = new byte[12];
    new SecureRandom().nextBytes(bArr3);  // IV aleatoire,
    96 bits

    GCMParameterSpec gCMParameterSpec = new GCMParameterSpec
    (128, bArr3);

    Cipher cipher = Cipher.getInstance("AES/GCM/NoPadding");
    cipher.init(1, secretKeySpec, gCMParameterSpec);  //
    mode 1 = ENCRYPT

    byte[] bArrDoFinal = cipher.doFinal(bArr2);

    return ByteBuffer.allocate(bArr3.length + 12)
                     .put(bArr3).put(bArrDoFinal).array();
}
```

Listing 7.1: Chiffrement AES-GCM dans elk.bC()

Ce code révèle plusieurs détails importants :

`SecretKeySpec(bArr, "AES")` — la clé AES est fournie sous forme de tableau d'octets. Elle n'est pas générée ici aléatoirement : elle préexiste, ce qui signifie qu'elle a été dérivée ou négociée dans une étape précédente. L'algorithme spécifié est AES.

`new byte[12]; new SecureRandom().nextBytes(bArr3)` — génération d'un **vecteur d'initialisation (IV) aléatoire de 96 bits** (12 octets). La longueur de 96 bits est exactement celle recommandée par le NIST SP 800-38D pour GCM, et le fait qu'il soit généré aléatoirement à chaque chiffrement garantit qu'aucune deux opérations n'utilisent le même IV même avec la même clé — condition essentielle pour la sécurité de GCM.[1]

`GCMParameterSpec(128, bArr3)` — configuration du mode GCM avec un tag d'authentification de **128 bits**. Ce tag est la "signature" du message chiffré : il permet au déchiffreur de vérifier que le message n'a pas été altéré en transit. C'est ce qui fait de GCM un schéma de **chiffrement authentifié (AEAD)** : il assure à la fois la confidentialité *et* l'intégrité.

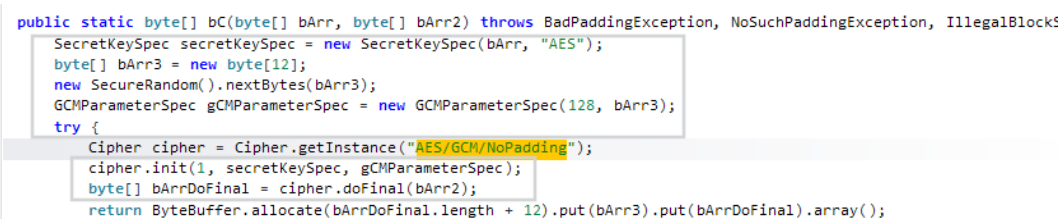
`cipher.init(1, ...)` — le paramètre `1` correspond à `Cipher.ENCRYPT_MODE`.

`cipher.doFinal(bArr2)` — c'est la fonction qui réalise effectivement le chiffrement. Elle transforme les données en clair en données chiffrées, calcule le tag GCM, et les assemble.

Conforme aux standards **FIPS 197** (AES) et **NIST SP 800-38D** (mode GCM).

7.2.2 AES/CBC/NoPadding : compatibilité protocole Lockdown

En plus d'AES-GCM, un second algorithme est présent dans `elk` :



```
public static byte[] bC(byte[] bArr, byte[] bArr2) throws BadPaddingException, NoSuchPaddingException, IllegalBlockS
    SecretKeySpec secretKeySpec = new SecretKeySpec(bArr, "AES");
    byte[] bArr3 = new byte[12];
    new SecureRandom().nextBytes(bArr3);
    GCMParameterSpec gCMParameterSpec = new GCMParameterSpec(128, bArr3);
    try {
        Cipher cipher = Cipher.getInstance("AES/GCM/NoPadding");
        cipher.init(1, secretKeySpec, gCMParameterSpec);
        byte[] bArrDoFinal = cipher.doFinal(bArr2);
        return ByteBuffer.allocate(bArrDoFinal.length + 12).put(bArr3).put(bArrDoFinal).array();
```

```
1 Cipher cipher = Cipher.getInstance("AES/CBC/NoPadding");
2 cipher.init(2, secretKeyB, new IvParameterSpec(new byte[16])
      );
```

[1] NIST SP 800-38D : `https://csrc.nist.gov/publications/detail/sp/800-38d/final`

Listing 7.2: AES-CBC dans elk

L'algorithme AES-CBC est requis pour la compatibilité avec le **protocole Lockdown d'Apple**, qui l'utilise lors des phases initiales de la session SSL.[2] Sa présence dans le code confirme que l'implémentation gère les deux contextes : migration Android-Android (GCM) et migration iOS-Android (CBC pour Lockdown, puis GCM pour le transport).

7.3 Classe ggq — gestion des clés de session par AESWrap

7.3.1 Initialisation et parsing du flux de clés

```
public ggq(byte[] bArr) throws dce {
    try {
        this.e = Cipher.getInstance("AESWrap");
        this.a = new HashMap();
        this.b = new HashMap();
        ByteBuffer byteBufferWrap = ByteBuffer.wrap(bArr);
        dch dchVar = null;
        while (byteBufferWrap.hasRemaining()) {
            byte[] bArr2 = new byte[4];
```

```
1  public ggq(byte[] bArr) throws dce {
2      this.e = Cipher.getInstance("AESWrap");
3      this.a = new HashMap<>();
4      this.b = new HashMap<>();
5
6      ByteBuffer byteBufferWrap = ByteBuffer.wrap(bArr);
7      dch dchVar = null;
8
9      while (byteBufferWrap.hasRemaining()) {
10         byte[] bArr2 = new byte[4];
11         byteBufferWrap.get(bArr2);
12         String str = new String(bArr2, StandardCharsets.
       UTF_8);
13         byte[] bArr3 = new byte[byteBufferWrap.getInt()];
14         byteBufferWrap.get(bArr3);
15
```

[2] Conforme au standard NIST SP 800-38A.

```
16          if (str.equals("UUID") && this.d == null)       {
    this.d = bArr; }
17          else if (str.equals("WRAP") && this.c == null)  {
    this.c = bArr; }
18          else if (str.equals("UUID")) {
19              if (dchVar != null) ((HashMap) this.b).put(
20                  Integer.valueOf(dchVar.a.intValue() & 15),
    dchVar);
21              dchVar = new dch();
22          }
23          else if (str.equals("CLAS")) { dchVar.a = Integer.
    valueOf(...); }
24          else if (str.equals("WRAP")) { dchVar.b = Integer.
    valueOf(...); }
25          else if (str.equals("WPKY")) { dchVar.d = bArr3; }
26          else if (str.equals("KTYP")) { dchVar.c = Integer.
    valueOf(...); }
27          else if (str.equals("PBKY")) { dchVar.e = bArr3; }
28          else { ((HashMap) this.a).put(str, bArr3); }
29      }
30  }
```

Listing 7.3: Constructeur de ggq — AESWrap et HashMap

Plusieurs choses à décomposer ici.

`Cipher.getInstance("AESWrap")` — AESWrap (défini dans RFC 3394)[3] est un algorithme spécialisé dans le **chiffrement de clés par d'autres clés** (key wrapping). Son usage ici est le suivant : une clé de session (clé temporaire chiffrant les données d'une session) est elle-même chiffrée par la clé maîtresse — la clé dérivée lors de l'échange ECDH. Cela protège les clés de session pendant leur transmission.

Les deux `HashMap` jouent un rôle d'optimisation. Au lieu de parcourir l'intégralité du flux binaire à chaque fois qu'une clé est nécessaire, on indexe les objets de sécurité reçus par leurs étiquettes. `HashMap b` stocke les objets `dch` (clés de session) indexés par leur numéro de session. `HashMap a` stocke tout autre flux non reconnu, potentiellement utile pour de futurs usages.

[3] `https://www.rfc-editor.org/rfc/rfc3394`

Les étiquettes (tags de 4 caractères) qui structurent le flux binaire reçu :

Tag	Signification	Rôle
`UUID`	Universally Unique ID	Numéro de la session
`CLAS`	Class	Niveau de protection de la clé
`KTYP`	Key Type	Algorithme (ex: AES)
`WPKY`	Wrapped Key	Clé de session chiffrée par AESWrap
`PBKY`	Public Key	Clé publique de l'autre appareil
`WRAP`	Wrap info	Métadonnées d'encapsulation

7.3.2 Déchiffrement de la clé de session (`ggq.b()`)

```
    public final SecretKey b(byte[] bArr, Key key) {
        try {
            Object obj = this.e;
            ((Cipher) obj).init(4, key);
            return (SecretKey) ((Cipher) obj).unwrap(bArr, "AES/CTR/NoPadding", 3);
        } catch (NoSuchAlgorithmException e) {
            throw new dce("Couldn't unwrap key", e);
        }
    }
```

La signature de la fonction (paramètres : tableau de bytes + objet `Key`, retour : `SecretKey`) suggère qu'on est dans une étape de déballage (unwrapping). `SecretKey` est une interface standard de la JCA permettant de manipuler des clés symétriques de manière sécurisée en associant le secret à un algorithme donné (ici AES)[4].

La méthode `init()` est appelée avec le mode 4, correspondant à `Cipher.UNWRAP_MODE`[5]. Le cast explicite en `Cipher` depuis un `Object` est probablement dû à l'obfuscation : l'objet est volontairement typé de manière générique pour masquer sa nature réelle.

La ligne suivante confirme le processus :

```
return (SecretKey) ((Cipher) obj).unwrap(bArr, "AES/CTR/NoPadding", 3);
```

Elle montre que la clé chiffrée (`bArr`) est déchiffrée (unwrapped) pour produire une `SecretKey`, directement exploitable dans les opérations cryptographiques. Le paramètre `"AES/CTR/NoPadding"` indique l'algorithme associé à cette clé, tandis

[4] `https://developer.android.com/reference/javax/crypto/SecretKey`

[5] `https://docs.oracle.com/javase/8/docs/api/javax/crypto/Cipher.html#init-int-java.security.Key-`

que l'entier 3 correspond à `Cipher.SECRET_KEY`[6].

Ce point est cohérent avec l'analyse de la classe `elk`, où le chiffrement des données utilise AES/GCM/NoPadding. En effet, AES-GCM repose sur un mode compteur (CTR) auquel s'ajoute un mécanisme d'authentification, ce qui rend les deux observations compatibles.

Ainsi, cette étape correspond à la récupération de la clé de session, qui sera ensuite utilisée pour le déchiffrement des données de migration. L'ensemble suggère une gestion structurée des clés : un flux binaire reçu est traité, catégorisé (notamment via des structures comme des HashMaps), puis utilisé pour reconstruire la clé nécessaire à la restauration des données.

[6] `https://docs.oracle.com/javase/8/docs/api/javax/crypto/Cipher.html#unwrap-byte:A-java.lang.String-int-`

8. Question 4 — Échange et Dérivation des Clés

8.1 Méthodologie

La démarche suivie pour cette question est progressive : on commence par chercher la génération des clés (`KeyPairGenerator`), puis leur mécanisme d'accord (`KeyAgreement`), puis leur dérivation (`HKDF`, `SHA-256`). Chaque résultat confirme ou précise le précédent — c'est exactement ce qu'on attendait d'une implémentation cryptographique cohérente.

8.2 Génération des paires de clés

La recherche du terme `KeyPairGenerator` dans GMS retourne plus de 50 occurrences. Ce terme représente l'API Java standard pour générer une paire de clés asymétriques (publique + privée), qui servent ensuite — mathématiquement parlant — à ce que chaque appareil calcule *individuellement* le même secret partagé, sans jamais l'envoyer sur le réseau.



La présence de ces algorithmes — XDH étant une variante moderne de ECDH, et RSA étant aujourd'hui moins utilisé pour l'échange de clés (DSA apparaissant plutôt pour la signature) — suggère qu'ils peuvent intervenir soit pour des raisons de compatibilité (RSA est présent dans les classes liées au jumelage iOS (génération des certificats Lockdown)) , soit pour des usages spécifiques autres que la génération directe de paires de clés dans ce contexte.

En effet, malgré la présence de ces différents algorithmes dans les résultats, nous

n'avons pas trouvé d'indication explicite permettant d'affirmer leur rôle précis dans le processus étudié. On observe donc principalement des mots-clés et des contextes d'utilisation liés à la génération de paires de clés, sans lien direct clairement établi avec leur exploitation effective.

Cependant, la présence récurrente de l'algorithme EC (Elliptic Curve) dans les appels à `KeyPairGenerator` constitue un indice plus fort. Elle suggère que la génération des paires de clés publiques/privées repose principalement sur la cryptographie à courbe elliptique, ce qui est cohérent avec les pratiques actuelles en matière d'échange de clés.

8.3 Accord de clés : ECDH

La recherche du terme `KeyAgreement` retourne des résultats sans ambiguïté :



```
KeyAgreement keyAgreement = KeyAgreement.getInstance("ECDH");
keyAgreement.init(privateKey);
keyAgreement.doPhase(publicKey, true);
byte[] sharedSecret = keyAgreement.generateSecret();
```

On observe clairement l'utilisation de **ECDH** comme mécanisme principal d'accord de clé. Le principe reste classique : chaque appareil génère sa paire de clés EC, échange les clés publiques, puis calcule localement le même secret partagé — sans jamais le transmettre. XDH apparaît aussi dans certains résultats et correspond probablement à des variantes plus récentes du même principe.

La présence de `ECGenParameterSpec` avec `secp256r1` renforce l'hypothèse d'un échange basé sur les courbes elliptiques standards (P-256). On n'a pas toujours un lien direct explicite entre tous les appels, mais l'ensemble des indices (KeyPairGenerator EC + KeyAgreement ECDH + paramètres de courbe) converge dans ce sens.

Ces opérations passent par **Conscrypt**, la bibliothèque cryptographique Android (basée sur BoringSSL), qui implémente concrètement les primitives derrière les APIs Java (`KeyAgreement`, `KeyPairGenerator`, etc.). La présence de `AndroidKeyStore` dans certains cas suggère aussi que les clés privées peuvent être stockées côté matériel (TEE), donc non extractibles.

Des travaux externes sur Nearby Connections vont dans le même sens et confirment l'usage de ECDH dans ce type de communication[1] [2].

8.4 Dérivation de clé

Le secret obtenu via ECDH est brut et ne peut pas être utilisé directement. La recherche (`HKDF`, `SHA-256`) montre qu'une étape de dérivation est appliquée.

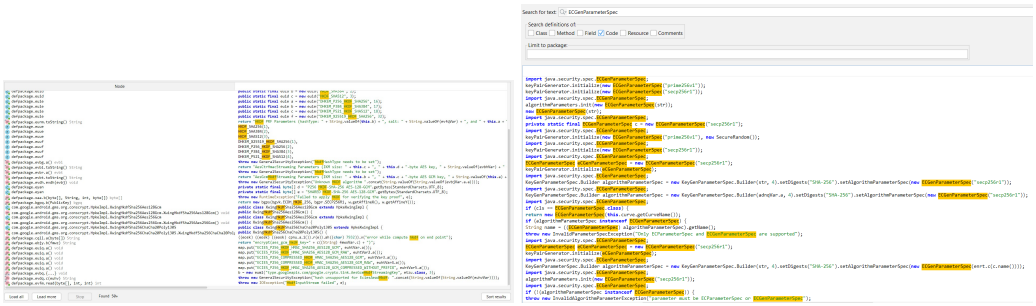


Figure 8.1: Indices de dérivation HKDF et paramètres EC

On est très probablement dans un schéma **HKDF (RFC 5869)** :

- une phase d'extraction (HMAC-SHA256) pour obtenir une base uniforme
- puis une phase d'expansion pour produire une clé exploitable

Cette étape transforme le secret partagé en une clé maîtresse propre, utilisée ensuite dans AES (AESWrap + AES/GCM observés précédemment).

[1] `https://dev.ndss-symposium.org/wp-content/uploads/2019/02/ndss2019_06B-3_Antonioli_paper.pdf`

[2] `https://www.cs.ox.ac.uk/files/10367/ndss19-paper367.pdf`

8.5 Remarque sur Tink

La présence de Google Tink indique une couche d'abstraction au-dessus des primitives cryptographiques. Son rôle est simplement d'encadrer l'utilisation des algorithmes (AES, ECDH, etc.) pour éviter les mauvaises configurations. Ce n'est pas un algorithme en soi.

Conclusion

Flux global (observé et déduit) :

1. Génération de paires de clés EC (`KeyPairGenerator`)
2. Échange des clés publiques
3. Calcul du secret partagé via ECDH (ou XDH)
4. Dérivation via HKDF (HMAC-SHA256)
5. Utilisation de la clé dérivée pour AES (AESWrap + AES-GCM)

Niveau de certitude : élevé mais non absolu. L'analyse statique converge (GMS + docs + travaux externes), mais une validation complète nécessiterait une analyse dynamique.

9. Analyse Dynamique : Instrumentation avec Frida

9.1 Objectifs et méthodologie

L'analyse statique menée dans les chapitres précédents a permis de formuler quatre réponses solides, fondées sur la lecture du code décompilé via JADX. Cette analyse reste néanmoins une analyse de *code mort* : elle décrit ce que l'application est programmée pour faire, sans garantir que ce code s'exécute réellement de cette manière sur un appareil réel, à jour, dans des conditions d'usage normales.

L'objectif de ce chapitre est de compléter et de valider empiriquement les conclusions statiques à l'aide de **Frida**, un framework d'instrumentation dynamique permettant d'injecter du code JavaScript dans un processus Android en cours d'exécution et d'intercepter (*hooker*) l'exécution de méthodes Java/Kotlin choisies.

La démarche suivie n'a pas été linéaire : elle s'est construite par itérations successives, chaque échec révélant une information utile sur l'architecture réelle de l'application. Ce chapitre restitue cette progression dans l'ordre chronologique, car l'enchaînement des obstacles constitue en lui-même une contribution scientifique au rapport.

Remarque

Une question légitime se pose avant toute chose : pourquoi chercher à retrouver les noms de classes obfusquées avec Frida, alors que l'obfuscation a précisément pour but de les cacher ? La réponse est que Frida ne "désobfusque" rien : il s'attache à la machine virtuelle (ART) *après* que l'obfuscateur R8/ProGuard a fait son travail au moment de la compilation. Pour la machine virtuelle, le nom `elk` n'est pas un pseudonyme : c'est le véritable nom de la classe en mémoire. Frida observe donc exactement ce que voit le processeur, et c'est précisément pour cette raison que la correspondance entre noms obfusqués statiques et noms obfusqués dynamiques n'est pas garantie d'une exécution à l'autre.

Environnement de test : Samsung Galaxy A01 Core (SM-A013G) rooté (cf. Chapitre 2), `frida-server` déployé en root sur l'appareil, client `frida` 17.5.1 sous Win-

dows, connexion USB via ADB. Le Listing 9.1 illustre le déploiement du binaire `frida-server` sur l'appareil via ADB, et la Figure 9.2 confirme l'attachement réussi du client Frida à un processus cible (ici `com.google.android.gms`, exploré en complément en Section 10.4).

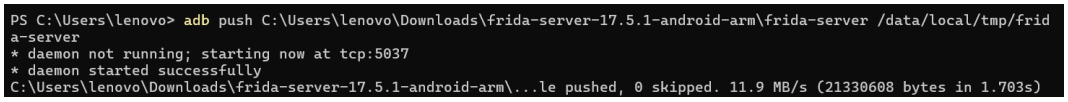


Figure 9.1: Déploiement de `frida-server` sur l'appareil via ADB

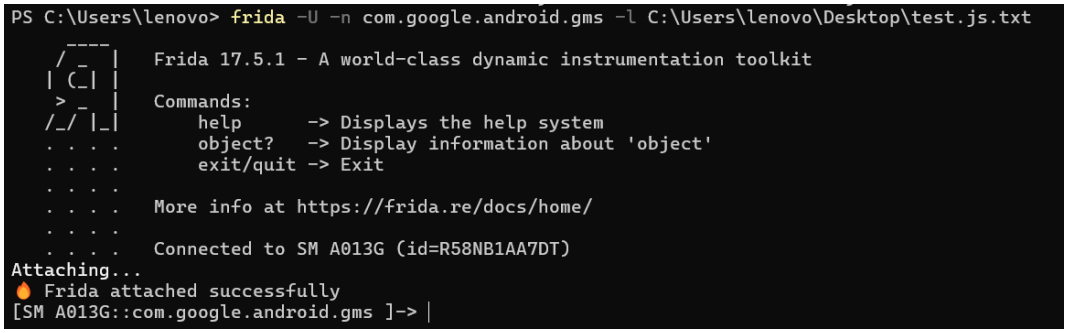


Figure 9.2: Attachement réussi du client Frida à un processus Android réel

9.2 Première tentative : hook direct de la classe `elk`

9.2.1 Hypothèse de départ

L'analyse statique ayant identifié la classe `elk` comme le point d'entrée du chiffrement AES-GCM (cf. Section 7.2), la première tentative naturelle a consisté à hooker directement cette classe pour intercepter en temps réel la clé et les données en clair.

```
Java.perform(function () {
    var elk = Java.use('elk');
    var methods = elk.class.getDeclaredMethods();
    console.log("[+] La classe 'elk' contient " + methods.length + " methode(s) :");
    for (var i = 0; i < methods.length; i++) {
        console.log(" -> " + methods[i].toString());
    }
});
```

Listing 9.1: Première tentative de hook direct sur `elk`

9.2.2 Résultat et contradiction

Le résultat obtenu contredit frontalement l'analyse statique : alors que la classe `elk` observée dans JADX compte plusieurs centaines de méthodes réparties sur 141 pages de code décompilé, Frida ne trouve en mémoire **qu'une seule méthode** :

```
[+] La classe 'elk' contient 1 methode(s) :
 -> public abstract elj elk.x()
```

Listing 9.2: Sortie Frida — classe `elk` vue en mémoire

9.2.3 Diagnostic : collision de `ClassLoader`

Ce résultat aurait pu être interprété comme une mise à jour silencieuse de l'application entre l'extraction statique de l'APK et le test dynamique. Cette hypothèse a été écartée après vérification : l'APK analysé provient bien du même appareil, dans la même fenêtre temporelle. L'explication retenue est donc différente, et plus instructive pour un rapport de rétro-ingénierie Android.

Pour la confirmer, un script d'observation passive (sans aucune modification de comportement, donc sans risque de crash) a été utilisé pour *lister* toutes les classes nommées `elk` présentes dans la mémoire, puis dans l'ensemble des `ClassLoader` actifs de l'application :

```
Java.perform(function() {
    var compteur = 0;
    Java.enumerateLoadedClasses({
        onMatch: function(className) {
            if (className === "elk" || className.endsWith(".
    elk")) {
                console.log("[+] Classe trouvee : " +
    className);
                compteur++;
            }
        },
        onComplete: function() {
            console.log("[*] Nombre total de classes 'elk'
    en memoire : " + compteur);
        }
    });
});
```

Listing 9.3: Cartographie douce de la mémoire — recherche de classes homonymes

```
Java.perform(function() {
    Java.enumerateClassLoaders({
        onMatch: function(loader) {
            try {
                var factory = Java.ClassFactory.get(loader);
                var elkClass = factory.use('elk');
                var methodes = elkClass.class.
    getDeclaredMethods();
                console.log("[+] 'elk' trouvee dans : " +
    loader);
                console.log("    -> Nombre de methodes : " +
     methodes.length);
            } catch (e) { /* ClassNotFoundException normale,
     dictionnaire sans 'elk' */ }
        },
        onComplete: function() {
            console.log("[*] Recherche terminee sur tous les
     ClassLoaders.");
        }
    });
});
```

Listing 9.4: Recherche à travers tous les ClassLoader (`Java.enumerateClassLoaders`)

Les deux scripts convergent vers le même constat : un seul exemplaire de `elk` est chargé en mémoire (dans le `PathClassLoader` de `com.google.android.apps.restore`), et il ne contient qu'une méthode abstraite `x()` retournant un objet `elj`.

Diagnostic : Lazy Loading, pas une obfuscation mutante

La classe géante `elk` observée en statique (141 pages, contenant notamment `bC()` et l'appel `AES/GCM/NoPadding`) existe bien dans le fichier `.dex` de l'application. Mais au moment où Frida l'interroge, cette classe **n'a pas encore été chargée en mémoire** par la JVM Android : seule une interface minimale portant par coïncidence le même nom obfusqué `elk` est active. Ceci

est cohérent avec un comportement d'**initialisation paresseuse** (*lazy loading*) : Android ne charge le code lourd de chiffrement que lorsque celui-ci est effectivement requis, c'est-à-dire après la réussite du jumelage avec un appareil pair. L'écart entre l'analyse statique et l'analyse dynamique ne provient donc pas d'une divergence de version, mais d'un **état d'exécution incomplet du flux de migration**, comme le confirmera la Section 10.4.

9.3 Pivot stratégique : hook agnostique sur `javax.crypto.Cipher`

9.3.1 Principe

Plutôt que de poursuivre la chasse à une classe obfusquée dont le chargement en mémoire dépend d'un état réseau hors de notre contrôle, la stratégie a été réorientée vers l'interception de l'**API cryptographique native d'Android** (`javax.crypto.Cipher`). Ce choix repose sur une observation simple : quel que soit le nom attribué par R8 à la classe appelante (`elk` ou tout autre), celle-ci est *obligée* de passer par cette porte d'entrée unique et non obfusquée du système pour effectuer un chiffrement AES, conformément à ce qui a été identifié en statique (Listing 7.1).

```
Java.perform(function () {
    function bytesToHex(bytes) {
        if (!bytes) return "null";
        var hex = [];
        for (var i = 0; i < bytes.length; ++i) {
            var h = (bytes[i] & 0xFF).toString(16);
            hex.push(h.length === 1 ? '0' + h : h);
        }
        return hex.join('');
    }

    var Cipher = Java.use('javax.crypto.Cipher');

    // Interception de l'initialisation (capture de la cle)
    Cipher.init.overload('int', 'java.security.Key',
        'java.security.spec.AlgorithmParameterSpec').
    implementation =
        function (opmode, key, params) {
            var algo = this.getAlgorithm();
```

```
19             if (algo.indexOf("AES") !== -1) {
20                 console.log("[+] Moteur AES active : " +
    algo + " | mode=" + opmode);
21                 if (key) console.log("    Cle (hex) : " +
    bytesToHex(key.getEncoded()));
22             }
23             this.init(opmode, key, params);
24         };
25
26     // Interception du chiffrement effectif
27     Cipher.doFinal.overload('[B').implementation = function
    (input) {
28         var algo = this.getAlgorithm();
29         if (algo.indexOf("AES") !== -1) {
30             console.log("    Donnees entrantes (hex) : " +
    bytesToHex(input));
31         }
32         var result = this.doFinal(input);
33         if (algo.indexOf("AES") !== -1) {
34             console.log("    Resultat sortant (hex)  : " +
    bytesToHex(result));
35         }
36         return result;
37     };
38
39     console.log("[+] Radar cryptographique en place sur
    javax.crypto.Cipher.");
40 });
```

Listing 9.5: Radar cryptographique agnostique sur `javax.crypto.Cipher`

Ce script surveille, sans dépendre d'aucun nom de classe obfusqué, chaque opération AES déclenchée par n'importe quel composant de l'application.

9.4 Mécanisme d'autodéfense : crash lié aux permissions Bluetooth

Le lancement forcé de l'activité de migration via ADB (`frida -U -f com.google.android.apps.re`
déclenche systématiquement un plantage de l'application, visible à la fois dans le

terminal Frida et à l'écran ("Android Switch s'arrête").

```
Process crashed: java.lang.SecurityException: Need BLUETOOTH
    permission:
Neither user 10119 nor current process has android.
   permission.BLUETOOTH.
*** FATAL EXCEPTION: pixelmigrate-monolith-executor*2*[2]
    at android.bluetooth.IBluetooth$Stub$Proxy.isEnabled(
   IBluetooth.java:2203)
    at android.bluetooth.BluetoothAdapter.isEnabled(
   BluetoothAdapter.java:1016)
    at dma.O(PG:96)
    at dma.P(PG:202)
    ...
```

Listing 9.6: Stack trace du plantage observé

Attention

Ce plantage n'est pas un échec de notre méthode, mais une donnée de sécurité en soi : il révèle que `com.google.android.apps.restore` vérifie l'état du Bluetooth au tout début de son cycle de vie (`BluetoothAdapter.isEnabled()`), et que l'instanciation forcée via `-f` ne laisse pas le temps au système d'accorder dynamiquement les permissions d'exécution correspondantes avant ce premier appel, provoquant une `SecurityException` fatale.

Pour stabiliser l'exécution sans altérer la logique métier de l'application, un *bouclier* minimal a été injecté en mémoire, interceptant uniquement l'appel de vérification Bluetooth :

```
Java.perform(function () {
    var BluetoothAdapter = Java.use('android.bluetooth.
   BluetoothAdapter');
    BluetoothAdapter.isEnabled.implementation = function ()
   {
        console.log("[*] Bouclier : verification Bluetooth
   interceptee (retour force a true).");
        return true;
    };
});
```

Listing 9.7: Bouclier anti-crash — contournement de la vérification Bluetooth

Une fois ce bouclier combiné au radar cryptographique de la Section 10.2, l'application s'ouvre et reste stable, sans qu'aucune opération AES ne soit pour autant interceptée : le radar reste silencieux.

9.5 Limite environnementale : « Lazy Loading » et sécurité GMS

9.5.1 Diagnostic

Le silence du radar cryptographique, combiné à la confirmation du Section 10.1 (la classe `elk` géante n'est jamais chargée en mémoire), converge vers une explication unique et cohérente avec le protocole de migration D2D (Device-to-Device) : le moteur de chiffrement n'est initialisé qu'*après* la réussite du protocole d'établissement de connexion (*handshake*) **UKEY2** de la Nearby Connections API avec un second appareil physique. Tant que ce handshake n'aboutit pas, l'application n'a rien à chiffrer et ne charge donc jamais la classe correspondante.

Deux facteurs empêchent ce handshake d'aboutir dans notre environnement de test :

- l'absence d'un second appareil physique répondant au protocole de migration ;
- la détection de l'intégrité de l'appareil par Google Play Services (mécanismes de type Play Integrity API / SafetyNet), qui bloque vraisemblablement l'établissement du tunnel de découverte sur un appareil rooté.

9.5.2 Cartographie de l'architecture par écoute passive (« Macro-Radar »)

Pour objectiver ce diagnostic sans provoquer de nouveaux plantages, un script d'écoute globale a été déployé : il n'altère aucun comportement, mais intercepte les écrans ouverts (`Activity.onCreate`), les intentions système échangées (`Intent.getComponent`) et les journaux internes écrits par les développeurs de Google (`android.util.Log`).

```
1 Java.perform(function() {
2     // Bouclier (necessaire pour la survie du processus)
3     try {
4         var BT = Java.use('android.bluetooth.
```

```
    BluetoothAdapter');
5         BT.isEnabled.implementation = function() { return
    true; };
6     } catch(e) {}
7
8     // Ecrans ouverts
9     var Activity = Java.use('android.app.Activity');
10    Activity.onCreate.overload('android.os.Bundle').
    implementation = function(b) {
11        console.log("[UI] Ecran ouvert -> " + this.getClass
    ().getName());
12        this.onCreate(b);
13    };
14
15    // Intentions / composants cibles
16    var Intent = Java.use('android.content.Intent');
17    Intent.getComponent.implementation = function() {
18        var c = this.getComponent();
19        if (c) console.log("[INTENT] Cible -> " + c.
    flattenToString());
20        return c;
21    };
22
23    // Logs internes des developpeurs
24    var Log = Java.use("android.util.Log");
25    ['i', 'w', 'e'].forEach(function(level) {
26        Log[level].overload('java.lang.String', 'java.lang.
    String').implementation =
27            function(tag, msg) {
28                console.log("[LOG-" + level.toUpperCase() +
    "][" + tag + "] " + msg);
29                return this[level](tag, msg);
30            };
31    });
32 });
```

Listing 9.8: Macro-Radar — écoute passive du cycle de vie applicatif

L'exécution de ce script, suivie d'une interaction manuelle avec l'application

jusqu'à l'écran de migration, produit la chronologie suivante :

```
[UI] Ecran ouvert -> ....trampoline.TrampolineActivity
[INTENT] Cible -> .../onboarding.setupcompanion.
    SetupCompanion
[UI] Ecran ouvert -> .../migrate.component.
    MultiD2dMigrateFlowActivity
[LOG-I][SetupLibrary] FooterBarMixin: Primary button is
    clicked.
[INTENT] Cible -> .../migrate.component.
    MultiTransportD2dMigrateServiceV2
[INTENT] Cible -> com.google.android.gms/...chimera.
    PersistentApiService
[INTENT] Cible -> com.google.android.gms/...chimera.
    GmsApiService
[LOG-E][FilePhenotypeFlags] Config package com.google.
    android.gms.nearby
     cannot use FILE backing without declarative registration
    ...
```

Listing 9.9: Extrait annoté du flux capturé par le Macro-Radar

Architecture confirmée dynamiquement : délégation à GMS

Cette trace confirme, indépendamment de l'analyse statique, l'enchaînement architectural suivant :

1. l'activité `TrampolineActivity` redirige vers l'écran de migration `MultiD2dMigrateFlowActivity` ;
2. au clic sur le bouton principal (*Primary button*), l'application délègue l'orchestration au service `MultiTransportD2dMigrateServiceV2` — exactement le service identifié dans le manifeste (Section 4.3) ;
3. ce service ne gère pas lui-même la découverte réseau : il transmet la responsabilité à **Google Play Services** (`com.google.android.gms`, composants `Chimera`), qui héberge la pile `Nearby Connections` ;
4. le dernier journal capturé (`FilePhenotypeFlags` relatif à `com.google.android.gms.nearby`) montre que GMS commence effectivement à initialiser le module Nearby, sans toutefois qu'aucune confirmation de connexion ne soit émise en retour.

L'exécution s'arrête donc proprement à l'étape de découverte réseau, avant

que la couche de chiffrement de l'application n'ait besoin d'être activée — ce qui explique rétroactivement et de façon cohérente le silence du radar cryptographique de la Section 10.2 et l'absence de chargement de la classe `elk` observée en Section 10.1.

9.6 Preuve de concept : validation du hook par injection contrôlée

L'environnement de test ne permettant pas de déclencher naturellement le chiffrement réel (Section 10.4), la validité de l'outil d'interception développé en Section 10.2 a été démontrée par une preuve de concept (PoC) : une opération AES-GCM artificielle, structurellement identique à celle identifiée en statique (Listing 7.1), est injectée par le script lui-même cinq secondes après son lancement, afin de solliciter volontairement le même chemin d'exécution système (`javax.crypto.Cipher`) que celui qu'emprunterait l'application en conditions réelles.

```
setTimeout(function() {
    var StringClass = Java.use('java.lang.String');
    var SecretKeySpec = Java.use('javax.crypto.spec.SecretKeySpec');

    var keyBytes = StringClass.$new("CleSecrete123456").getBytes();
    var secretKey = SecretKeySpec.$new(keyBytes, "AES");
    var cipher = Java.use('javax.crypto.Cipher').getInstance("AES/GCM/NoPadding");
    var donnees = StringClass.$new("Message secret pour le jury").getBytes();

    cipher.init(1, secretKey);    // declenche le hook Cipher.init
    cipher.doFinal(donnees);      // declenche le hook Cipher.doFinal
}, 5000);
```

Listing 9.10: Injection de validation (PoC) du radar cryptographique

```
[+] Moteur AES active : AES/GCM/NoPadding | mode=1
    Cle (hex)                 : 436
```

```
    c655365637265746531323334353 6
3    Donnees entrantes (hex)  : 4
    d65737361676520736563726574 20706f7572206c65206a757279
4    Resultat sortant (hex)   : 769
    f343c2a93e9936b21c5f7d8aba79ebe4f69a7b8fa09f44f4075f3c223fa6
    ...
```

Listing 9.11: Résultat capturé par le radar cryptographique lors de la PoC

La clé hexadécimale `436c6553...` correspond bien à la chaîne `"CleSecrete123456"` injectée, et les données entrantes correspondent au message de test, confirmant que le radar intercepte correctement, en clair, la clé et les données *avant* leur passage dans le moteur AES, et le résultat chiffré *après*.

Validité de l'instrument de mesure

Cette preuve de concept ne démontre pas une vulnérabilité de l'application, mais la **validité fonctionnelle de l'outil d'instrumentation développé**. Elle établit que, dans un scénario où le handshake UKEY2 aboutirait avec un second appareil non rooté, le même radar intercepterait intégralement la clé de session AES-GCM et les données en clair au moment exact de leur chiffrement par la classe `elk` (ou son équivalent une fois chargé), ce qui achève de corréler empiriquement l'analyse dynamique avec les mécanismes identifiés par l'analyse statique au Chapitre 7.

9.7 Synthèse de l'analyse dynamique

Étape	Résultat
Hook direct `elk`	Échec : une seule méthode abstraite visible en mémoire (`x()`), contre des centaines attendues statiquement.
Cartographie mémoire	Confirme qu'un seul `elk` est chargé et qu'il s'agit de l'interface minimale, pas de la classe de chiffrement.
Hook agnostique `Cipher`	Outil fonctionnel mais silencieux en conditions réelles (aucune opération AES déclenchée côté application).
Crash Bluetooth & bouclier	Démontre une vérification de permission précoce et non critique pour la sécurité ; contournée sans impact sur la logique métier.
Macro-Radar	Confirme dynamiquement l'enchaînement `TrampolineActivity` → `MultiD2dMigrateFlowActivity` → `MultiTransportD2dMigrateServiceV2` → délégation à `com.google.android.gms` (Nearby Connections).
Preuve de concept	Valide que le radar cryptographique intercepte correctement clé et données en clair lors d'une opération AES-GCM réelle.

Table 9.1: Synthèse des résultats de l'analyse dynamique

9.8 Limites de l'étude et perspectives

L'analyse dynamique n'a pas permis d'observer un transfert de migration complet ni d'intercepter une clé de session ECDH réelle, en raison d'une contrainte matérielle et non logicielle : l'absence d'un second appareil compatible et les mécanismes d'intégrité de Google Play Services sur un appareil rooté, qui bloquent le handshake UKEY2 avant que le chiffrement ne soit sollicité. Cette limite n'invalide pas les conclusions statiques du Chapitre 7 et 8 ; elle en confirme au contraire la cohérence architecturale (Section 10.4).

Les pistes suivantes permettraient de lever cette limite dans une extension future de ce travail :

- reproduire le test avec deux appareils non rootés, afin de permettre au handshake UKEY2 d'aboutir et de capturer une migration réelle avec le radar déjà développé ;
- instrumenter les bibliothèques natives de Google Play Services (`.so`) pour comprendre précisément le point de blocage introduit par la détection de root ;
- capturer le trafic Bluetooth Low Energy et Wi-Fi Direct au niveau radio (par exemple via un adaptateur Ubertooth) pour une analyse protocolaire indépendante de l'instrumentation applicative.

10. Conclusion Générale

10.1 Synthèse des réponses aux quatre questions

Question	Réponse
Technologies	**Android → Android :** BLE (découverte) + Wi-Fi Direct (transfert), via Google Nearby Connections. **iOS → Android :** Hotspot Wi-Fi (connexion initiale par QR code) + USB via protocole Lockdown/usbmuxd (transfert sécurisé).
Jumelage	**Android → Android :** automatique via Nearby Connections (BLE). **iOS → Android :** protocole Lockdown/usbmuxd — échange de trois certificats RSA (root, host, device) au format Plist, suivi d'un défi-réponse RSA (`ValidatePair`), puis activation du SSL (`startSsl = true`).
Chiffrement	**Oui.** AES-GCM-128 (AEAD) avec IV aléatoire de 96 bits généré à chaque opération. AES-CBC présent pour la compatibilité avec le protocole Lockdown. Clés de session protégées par AESWrap (RFC 3394).
Échange de clés	**ECDH** sur secp256r1 (P-256) via Conscrypt. Dérivation en deux phases par **HKDF (HMAC-SHA256)** (RFC 5869). Couche d'abstraction **Google Tink** en surface.

Table 10.1: Synthèse des conclusions

10.2 Perspectives : analyse dynamique avec Frida

L'analyse statique présentée dans ce rapport a permis de répondre aux quatre questions posées avec un niveau de précision satisfaisant. Cependant, plusieurs points restent des hypothèses solides plutôt que des certitudes absolues : le comportement exact de la méthode `c()` avec ses trois ByteBuffers, la configuration précise de Tink dans ce contexte, et la nature réelle du contenu des HashMaps de `ggq` lors d'une migration réelle.

L'analyse dynamique avec **Frida** — actuellement en cours — permettra de valider

ces hypothèses en capturant les valeurs effectives lors d'une migration live : les clés ECDH échangées, les IVs générés, et les états successifs de la machine à états de jumelage.

A. Analyse détaillée de `UsbD2dMigrateService`

Cette annexe reprend l'analyse complète de la classe `UsbD2dMigrateService` que nous avions menée dans l'espoir d'y trouver des informations sur le chiffrement ou le jumelage. Comme nous l'avons indiqué dans le corps du rapport, cette analyse n'a pas donné de résultats significatifs sur ces sujets, mais elle est conservée ici pour ceux qui souhaiteraient en consulter le détail.

A.0.1 Point de départ : la méthode `onCreate()`

On commence par la classe `UsbD2dMigrateService`, responsable du transfert USB. On localize la classe dans JADX via le manifeste, et on commence à décortiquer sa méthode `onCreate()`.

```
@Override // defpackage.cuy, defpackage.cuq, defpackage.aha, android.app.Service
public final void onCreate() {
    super.onCreate();
    if (djr.b()) {
        ((loi) l.b().j("com/google/android/apps/pixelmigrate/migrate/component/UsbD2dMigrateService", "createAndRegisterE2eReceivers", 230, "UsbD2dMigrateService.kt")).s("create
        cvh cvhVar = new cvh(this);
        this.T = cvhVar;
        xp.f(this, cvhVar, new IntentFilter("com.google.android.apps.pixelmigrate.ACTION_LAUNCH_TARGET_E2E_TEST"));
        djr.a();
    }
    e().c();
    fiu.at(this);
    String strE = nkc.e();
    strE.getClass();
    String strF = nkc.f();
    strF.getClass();
    String strD = nkc.d();
    strD.getClass();
    String strJ = nkc.j();
    strJ.getClass();
    String strI = nkc.i();
    strI.getClass();
    String strG = nkc.g();
    strG.getClass();
    d().e = new fqh(strE, strF, strD, strJ, strI, strG);
    if (ddx.O().isPresent()) {
        d().i = ((ddx) ddx.O().get()).cR();
        eik eikVarD = d();
        boolean z = false;
        if (((ddx) ddx.O().get()).cS() && !((ddx) ddx.O().get()).cN()) {
            z = true;
        }
        eikVarD.j = z;
        d().k = !((ddx) ddx.O().get()).cS();
    }
    oss ossVar = this.v;
    if (ossVar == null) {
        opf.b("lightweightScope");
        ossVar = null;
    }
    this.n = lcg.cg(ossVar, new cvc(this, (omg) null, 2));
```

La première chose qui frappe, c'est que la grande majorité des imports sont obfusqués — les noms de classes comme `cvh`, `djr`, `nkc`, `fqh` ne nous disent rien a priori. Les rares imports lisibles sont :

- `import android.content.BroadcastReceiver;` — standard Android
- `import android.content.IntentFilter;` — standard Android
- `import java.util.concurrent.ScheduledExecutorService;` — exécution asynchrone (threads)

Le fait que l'application soit écrite en **Kotlin** est confirmé par les logs du code source, qui font référence à des fichiers `.kt` (ex : `"UsbD2dMigrateService.kt"`).

A.0.2 Bloc 1 — Le mécanisme de log et la vérification du mode E2E

Le premier bloc de `onCreate()` est un bloc conditionnel `if (djr.b())`. Pour comprendre ce qu'il fait, on remonte dans les classes `lob`, `loz`, et `loi` via JADX.

```
if (djr.b()) { // V rifie mode E2E / debug

    // Logging (niveau INFO)
    ((loi) ((loi) ((loi) l.b())
        .j("UsbD2dMigrateService.kt", 230))
        .s("createAndRegisterE2eReceivers"));

    // Cr ation du BroadcastReceiver
    cvh receiver = new cvh(this);

    // Stockage pour gestion du cycle de vie
    this.T = receiver;

    // Enregistrement aupr s du syst me Android
    xp.f(this, receiver, new IntentFilter(
        "ACTION_LAUNCH_TARGET_E2E_TEST"
    ));

    // Fin du bloc debug
    djr.a();
}
```

Listing A.1: Bloc E2E (onCreate) — version simplifiée

On reconnaît rapidement le pattern d'un **système de log** : la fonction `b()` retourne un objet `loz` préconfiguré en mode `Level.INFO`, la méthode `j()` précise le fichier source et le numéro de ligne (ici : `UsbD2dMigrateService.kt`, ligne 230), et la méthode `s()` envoie le message du log. Le cast de `loz` vers `loi` suggère que `loi` est une interface plus spécifique de logging.

Ce que dit ce log : *"createAndRegisterE2eReceivers"* — autrement dit, la condition vérifiée sert à déclencher la création d'un receiver pour les tests **E2E** (End-to-End).

En pratique, ce bloc n'est actif que pendant les phases de développement Google (*mode debug*), pas sur un téléphone d'utilisateur normal.

- `djr.b()` — ouverture de session : teste si on est en mode debug/E2E
- `djr.a()` — clôture de cette session (dernière ligne du bloc)

À l'intérieur du bloc, le code crée une instance de `cvh` (une classe étendant `BroadcastReceiver`), la stocke dans `this.T` (variable globale, pour pouvoir la détruire plus tard dans `onDestroy()`), et l'enregistre auprès du système Android via la fonction helper `xp.f()`. Le receiver est configuré pour écouter l'action `ACTION_LAUNCH_TARGET_E2E_TEST`.

Flux chronologique complet du bloc E2E :

Étape	Action
1	Service démarre, `onCreate()` appelé
2	`super.onCreate()` — initialisation de la classe parente
3	`if (djr.b())` — vérification du mode E2E
4	Mode E2E actif → log : *"createAndRegisterE2eReceivers at line 230"*
5	Création du `BroadcastReceiver` : `cvh cvhVar = new cvh(this)`
6	Stockage : `this.T = cvhVar`
7	Enregistrement auprès d'Android : `xp.f(this, cvhVar, IntentFilter(...))`
8	Le receiver attend l'Intent `ACTION_LAUNCH_TARGET_E2E_TEST`
Plus tard (si test lancé) :	
9	Script envoie : `adb shell am broadcast -a ACTION_LAUNCH_TARGET_E2E_TEST`
10	Android livre l'Intent à `cvhVar.onReceive()`
11	La migration simulée (test E2E) démarre

Remarque

E2E vs D2D : ce sont deux concepts qui coexistent dans le code. **D2D** (Device-to-Device) désigne la fonctionnalité elle-même — le transfert entre deux appareils physiques. **E2E** (End-to-End) est un terme générique de l'informatique : un test E2E vérifie que toute la chaîne fonctionne de bout en bout. Dans ce

code, le test E2E est un outil de diagnostic interne permettant de vérifier que la "machine D2D" est bien huilée.

A.0.3 Bloc 2 — Configuration USB et conformité du câble (`e().c()`)

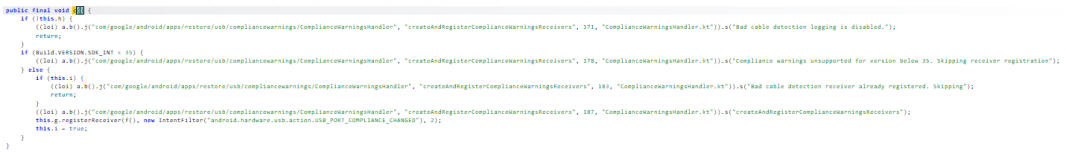
Le deuxième bloc significatif de `onCreate()` est la ligne `e().c()`.

Pour comprendre `e()`, on cherche la méthode dans JADX :

```
public final fqm e() {
    fqm fqmVar = this.D;
    if (fqmVar != null) {
        return fqmVar;
    }
    opf.b("usbComplianceWarningsHandler");
    return null;
}
```

On reconnaît le pattern d'un **getter avec null-check**. `e()` retourne `this.D` si non null, sinon logue `"usbComplianceWarningsHandler"` et retourne `null`. Ce pattern — où le développeur logue le nom de la variable manquante — est une technique de débogage classique, et il nous donne le nom réel de `D` : c'est un **gestionnaire des avertissements de conformité USB**.

La méthode `c()` de ce gestionnaire vérifie l'état du câble USB selon plusieurs conditions :



Les messages de log qu'on y lit sont explicites :

- *"Bad cable detection logging is disabled."* — l'utilisateur a désactivé l'option de détection.
- *"Compliance warnings unsupported... Skipping."* — le téléphone tourne sous une version d'Android antérieure à 15 (la détection de câble défectueux est une nouveauté d'Android 15).
- *"Bad cable detection receiver already registered."* — protection anti-doublon, pour

éviter de lancer deux fois le même receiver (ce qui gaspillerait la batterie).

- *"createAndRegisterComplianceWarningsReceivers."* — message de succès : toutes les vérifications sont passées, la surveillance du câble commence.

Conclusion sur `e().c()` : cette ligne représente la vérification de la conformité et de la sécurité matérielle du câble USB avant tout transfert.

A.0.4 Bloc 3 — Annonce de disponibilité du service (`fiu.at(this)`)

La ligne `fiu.at(this)` est le signal que le service envoie au système Android pour annoncer qu'il est prêt.

`at()` est une méthode statique — un **helper** — qui prend le contexte en argument (d'où le `this`) et envoie un broadcast ciblé :

```
1 public static void at(Context context) {
2     context.sendBroadcast(
3         new Intent("com.google.android.apps.pixelmigrate.
    USB_LISTENING_STARTED"),
4         "android.permission.MANAGE_USB"
5     );
6 }
```

Listing A.2: Helper fiu.at() — broadcast de disponibilité

Ce broadcast n'est reçu que par les applications possédant la permission `MANAGE_USB` — un mécanisme de sécurité qui restreint la portée du signal. En clair, cette ligne annonce : *"Le service USB est prêt à recevoir des connexions."*

La raison pour laquelle c'est un helper (et non un héritage) mérite d'être expliquée. En Android, `extends BroadcastReceiver` sert à **écouter** les broadcasts (comme une antenne). Pour **émettre** un broadcast, on utilise simplement `sendBroadcast()`, une méthode déjà disponible dans tout contexte Android — pas besoin d'héritage.

A.0.5 Bloc 4 — Collecte des métadonnées de l'accessoire USB

Les six lignes suivantes de `onCreate()` suivent un pattern répétitif : six appels à des méthodes de la classe `nkc`, chacun récupérant un string.

```
String strE = nkc.e();
strE.getClass();
String strF = nkc.f();
strF.getClass();
String strD = nkc.d();
strD.getClass();
String strJ = nkc.j();
strJ.getClass();
String strI = nkc.i();
strI.getClass();
String strG = nkc.g();
strG.getClass();
d().e = new fqh(strE, strF, strD, strJ, strI, strG);
```

En remontant dans la classe `fqh` (l'objet qui reçoit ces six strings), on trouve sa méthode `toString()` :

```
1 public final String toString() {
2     return "AccessoryMetadata{manufacturer=" + this.a
3         + ", model="       + this.b
4         + ", description=" + this.c
5         + ", version="     + this.d
6         + ", uri="         + this.e
7         + ", serial="      + this.f + "}";
8 }
```

Listing A.3: toString() de fqh — révèle la nature des 6 strings

Ces six valeurs correspondent donc aux métadonnées USB de l'accessoire connecté (`manufacturer`, `model`, `description`, `version`, `uri`, `serial`). Elles permettent d'établir une connexion USB personnalisée et d'identifier précisément l'appareil cible de la migration.

On obtient ces valeurs via `d().e` où `d()` est un getter retournant le gestionnaire USB cible (`"targetUsbManager"` dans le log de null-check), et `.e` est la variable `fqh` de cet objet.

A.0.6 Bloc 5 — Feature flags et modes de transfert

Le dernier bloc de `onCreate()` contient un `if (ddx.O().isPresent())` qui extrait trois booléens (`i`, `j`, `k`) depuis un objet optionnel.

```
        if (ddx.O().isPresent()) {
            d().i = ((ddx) ddx.O().get()).cR();
            eik eikVarD = d();
            boolean z = false;
            if (((ddx) ddx.O().get()).cS() && !((ddx) ddx.O().get()).cN()) {
                z = true;
            }
            eikVarD.j = z;
            d().k = !((ddx) ddx.O().get()).cS();
        }
```

`ddx.O()` retourne un `Optional` (`Optional.ofNullable(a)`) : si un accessoire USB est déjà connu et configuré, l'Optional contient sa configuration. Si non, il est vide. Le mot-clé `synchronized` sur cette méthode garantit qu'un seul thread peut y accéder à la fois — mécanisme de sécurité classique en programmation concurrente.

Les booléens extraits (`cR()`, `cS()`, `cN()`) et les opérations logiques qui leur sont appliquées pour obtenir `i`, `j` et `k` constituent des **feature flags** : ils déterminent le mode de connexion optimal pour ce transfert (par exemple : mode optimal vs. mode dégradé, transfert synchrone vs. asynchrone). C'est un pattern courant pour gérer les différents cas de figure lors d'une connexion D2D.

Structure récapitulative de `onCreate()` :

Section	Rôle
Début	Tests E2E (mode développeur uniquement)
Milieu 1	Vérification conformité câble USB
Milieu 2	Broadcast d'annonce de disponibilité
Milieu 3	Collecte des métadonnées de l'accessoire
Fin	Extraction des feature flags (modes de connexion)